

Prague University of Economics and Business

Faculty of Informatics and Statistics



EXPLORING PREDICTORS OF RESULTS OF DOTA 2 MATCHES

BACHELOR THESIS

Study programme: Data Analytics

Author: Gleb Tcypin

Supervisor: Ing. Jan Rejthar

Prague, June 2025

Acknowledgement

I want to thank my supervisor Ing. Jan Rejthar for his valuable advice and guidance during the thesis creation

Also, big gratitude and respect belongs to my older brother Anton for keeping me on the right track for a long time

Abstract

This thesis investigates the prediction of professional Dota 2 match outcomes using machine learning techniques, focusing on pre-game and early-game features limited to the first 10 minutes of gameplay. Alongside the development of predictive models, it introduces a structured data governance framework inspired by modern data engineering practices, enabling scalable data collection, transformation, and modelling.

Three classification algorithms are applied: Logistic Regression, Multi-Layer Perceptron (MLP), and XGBoost. Each model is trained on a dataset adjusted to suit its specific characteristics and working principles, incorporating custom transformations such as binning and one-hot encoding. The models undergo initial evaluation and are then refined through feature pruning and hyperparameter tuning. The impact of individual variables is assessed using model coefficients, permutation importance (measured on the training data), and Shapley values.

The models achieve competitive accuracy, with all reaching up to 65.7%. Although pruning and tuning yield incremental improvements, none of the individual models surpasses the baseline Logistic Regression, which also reaches 65.7%. A stacked ensemble model that combines the best-performing versions of all three base learners slightly improves performance to 65.8%.

Key predictors include early-game differences in experience, kills, and net worth, as well as the presence of a team on the ELO leaderboard. The results confirm that early-game data alone can support meaningful predictions and that combining ensemble techniques with structured preprocessing pipelines enhances both performance and interpretability. Future work may involve integrating real-time match data and team-specific historical information to further improve predictive accuracy.

Keywords

Dota 2, Machine Learning, XGBoost, Logistic Regression, Data Governance, MLP

Abstrakt

Tato práce se zabývá predikcí výsledků profesionálních zápasů ve hře Dota 2 pomocí metod strojového učení, přičemž se zaměřuje na předzápasové a rané herní ukazatele omezené na prvních 10 minut hry. Kromě samotné tvorby prediktivních modelů zavádí práce také strukturovaný rámec pro správu dat inspirovaný moderními přístupy datového inženýrství, který umožňuje škálovatelné sbírání, transformaci a modelování dat.

Jsou využity tři klasifikační algoritmy: logistická regrese, vícevrstvá neuronová síť (MLP) a XGBoost. Každý model je natrénován na datové sadě přizpůsobené jeho konkrétním vlastnostem a způsobu fungování, přičemž jsou použity úpravy jako je rozdělení hodnot do intervalů (binning) a jednorázové zakódování kategorií (one-hot encoding). Modely jsou nejprve vyhodnoceny v základní podobě a následně vylepšeny prostřednictvím redukce méně významných příznaků a ladění hyperparametrů. Vliv jednotlivých proměnných je posuzován pomocí koeficientů modelu, permutační důležitosti (vypočtené na trénovacích datech) a Shapleyho hodnot.

Modely dosahují srovnatelné přesnosti, přičemž všechny dosahují až 65,7 %. I když ladění a redukce příznaků přinesly dílčí zlepšení, žádný z jednotlivých modelů nepřekonal výchozí logistickou regresi, která rovněž dosáhla 65,7 %. Kombinovaný model (stacked ensemble), který propojuje nejlepší verze všech tří základních modelů, zlepšil výsledek mírně na 65,8 %.

Mezi nejvlivnější příznaky patří rozdíl v získaných zkušenostech, počtu zabití, čistém jmění týmů během prvních minut hry a také přítomnost týmu v žebříčku ELO. Výsledky potvrzují, že smysluplné predikce lze dosáhnout i pouze na základě raných dat ze hry a že kombinace ensemble metod s dobře strukturovaným předzpracováním zvyšuje jak výkonnost, tak interpretovatelnost modelů. Do budoucna by bylo možné zlepšit predikční přesnost začleněním dat v reálném čase a historických informací o konkrétních týmech.

Klíčová slova

Dota 2, strojové učení, XGBoost, logistická regrese, řízení dat, MLP

Contents

Introduction	7
1 Dota 2 overview.....	9
1.1 Game rules & main mechanics	9
1.2 The Prediction Problem in Dota 2.....	13
2 Data Architecture overview	15
3 Theoretical part	17
3.1 Evaluation metrics	17
3.1.1 Accuracy:	17
3.2 Machine learning models introduction	17
3.3 Hyperparameter optimization	19
3.3.1 Random Search.....	20
3.3.2 K-fold cross-validation	20
3.4 Data transformation techniques overview.....	21
3.4.1 One-hot encoding	21
3.4.2 Binning.....	21
3.5 T-test	22
3.6 Feature importance.....	22
3.6.1 SHAP values & beeswarm plot	22
3.6.2 Coefficient-based importance in Logistic Regression.....	24
3.6.3 Permutation-based importance in Neural Networks.....	24
4 Practical part	25
4.1 Dataset overview	25
4.2 Data transformations.....	26
4.2.1 Elo ratings.....	26
4.2.2 Tower destruction timings	26
4.2.3 Rest of string-type variables	27
4.2.4 <i>first_blood_time</i> transformation	27
4.3 Features exploring & model creation.....	27
5 Discussion.....	44
Conclusion	45
List of references	46
Appendices	I
A. SQL Scripts for tables creation.....	II

B.	Python script for data processing example	VII
C.	Endpoints list	XII
D.	Dataset preview	XIII
E.	Features description.....	XIV

Introduction

Nowadays, esports (also known as electronic sports) are becoming increasingly popular (*The Surprising History of Video Games and Esports*, 2024). The industry continues to attract more people through growing appeal, an expanding pool of competitive games, and the organization of many tournaments for a wide range of players—from amateurs to high-tier professionals. It has become common to see prize pools in esports tournaments surpass those of well-established traditional sports disciplines.

For example, *The International 2018* had a prize pool of \$25,532,177, significantly more than the *PGA Championship 2018*, which had \$11,000,000 (Yao, 2019).

Moreover, the intersection between esports and various economic sectors has become more frequent: medical institutions, payment service providers, logistics companies, and even military organizations have begun sponsoring esports events. For instance, logistics giant DHL and the U.S. Air Force were among the sponsors of the *Dota 2* tournament (*DreamLeague Season 24 - Winner - ESL Dota2 DreamLeague*, 2025).

It is increasingly evident that one does not need to be a professional player or even a gaming enthusiast to recognize that esports have become a part of everyday life. The esports industry has created numerous job opportunities that previously did not exist—such as esports event managers, content creators, professional players, coaches, and analysts (Perez, 2024).

Beyond the competitive scene, the influence of esports-like games on individuals' lives has also been examined. A recent study on the esports phenomenon surveyed college students across various academic levels, asking how competitive games affected their studies and overall well-being. Many participants reported positive impacts, including improved social interactions and enhanced critical thinking skills (Delello et al., 2021).

In the case of the Czech Republic, the *Czech Association of Esports* (*Česká Asociace Esportu*) reports that nearly 1.9 million Czechs have watched esports match at least once in their lives (SRO, 2022).

This thesis focuses on *Dota 2* as one of the key esports' disciplines. *The International 2021* holds the record for the highest prize pool in esports history. Furthermore, 8 out of the top 10 esports tournaments with the highest prize pools are related to *Dota 2* (Largest Overall Prize Pools in Esports - Esports Earnings, 2025).

From the perspective of audience engagement, *Dota 2* consistently ranks among the top 10 most viewed categories on Twitch from 2020 to 2024, with 542,038,197 hours watched in 2024 alone (Twitch Stats and Analytics - Games & Channels - SullyGnome, 2025).

This thesis aims to reflect on the process of building a complete data architecture designed to explore the key factors influencing the outcome of a *Dota 2* match. This architecture encompasses data extraction, transformation, and lineage, culminating in a machine

learning layer that allows for the construction of a robust model, evaluation of its performance, interpretation of results, and identification of the most influential features.

As a starting point, the main rules and concepts of *Dota 2* will be introduced to help the reader understand the data transformation process, factor analysis, model training and evaluation, and result interpretation.

Additionally, the reader will make yourself familiar with challenges of *Dota 2* match prediction and various analytical approaches to understanding game factors.

The primary goal of this thesis is to uncover the underlying logic of *Dota 2* gameplay using machine learning — specifically, to identify the key factors that most influence the final match outcome. Additionally, the thesis aims to demonstrate how data collection, feature engineering, and model development can be integrated into an agile and reusable framework. Such a framework offers valuable insights for analysing complex domains, such as esports match prediction.

1 Dota 2 overview

1.1 Game rules & main mechanics

Match Structure in Dota 2

A standard Dota 2 match features two teams: the Radiant and the Dire, each composed of five players. The primary goal is to destroy the enemy's Ancient, a heavily protected structure located deep within their base.

All matches take place on a single, standardized map, with no alternative layouts or changes in objective positions. While minor updates to the map may occur through official game patches, the overall layout remains consistent. Refer to Figure 1.1 and Table 1.1 to see important map spots.

Heroes and Abilities

Each player controls a hero, a unique character with distinct attributes and abilities. There are currently 126 heroes in the game. Every hero begins with base statistics like health, mana (used for casting abilities or using mana-dependent items), attack damage, armor, and magic resistance. These values improve as the hero levels up through gameplay.

Most heroes have four core abilities, but some may possess more. These include:

- Active abilities – manually triggered by the player
- Passive abilities – automatically apply effects once learned
- Ultimate ability – the hero's strongest skill, typically unlocked at levels 6, 12, and 18

Most abilities and items have a cooldown, which is the period of time that must pass before the ability or item can be used again.

Before the match begins, players also choose from two Facets, which modify or enhance gameplay in a unique way. This choice cannot be changed once the game starts. As the hero gains levels, players unlock Talents at levels 10, 15, 20, 25, and between 26–30. Talents offer branching improvements, such as increased stats, faster movement, or upgraded abilities.

Items and Inventory

Heroes use gold, the in-game currency, to purchase items that enhance their capabilities. Items are bought from the base shop, which is accessible while standing in the team's base or remotely via the courier. Players on the map—such as those fighting in lanes—can request the courier to deliver items directly to their current location.

Each player can carry up to six active items in their main inventory and three backup items in their backpack. Items in the backpack remain inactive unless moved into the main inventory. Just like abilities, items also have cooldowns after being used, can be either active (manual activation) or passive (no activation required) and can be mana demanding.

All heroes start the game with 600 gold, with one exception: the hero Alchemist can begin with more if using the “Seed Money” Facet.

Each player also controls a courier, a small artificially controlled unit that delivers items. If a courier is killed, it temporarily disables deliveries for that player and grants gold to the opposing team.

Pregame and First Blood

Matches begin with a 90-second pregame phase. During this time, players may purchase starting items, move across the map, or attempt early engagements. The first kill of the match is called First Blood and awards an extra 135 gold to the hero who achieves it. First Blood kill can occur after pregame phase.

Neutral Objectives

The map contains several neutral objectives—important targets that are not affiliated with either team:

- Roshan is the strongest neutral unit. When killed, it grants gold to the entire team and experience to nearby allies. Roshan may also drop special items, such as the *Aegis of the Immortal*, which allows a hero to revive once after death.
- Tormentor is a neutral structure that, when destroyed, grants an Aghanim’s Shard to a random eligible teammate. This item unlocks or upgrades a specific hero ability.

Lane Creeps and Lane Structure

Every 30 seconds starting from game time 00:00, each team spawns a wave of lane creeps—artificially controlled allied units that automatically march down the three primary paths: Top, Middle, and Bottom lanes.

Each lane includes:

- Three towers per team: Tier 1 (closest to the river with Active runes spawns – see Figure 1.1), Tier 2, and Tier 3
- Two barracks per team: one for melee creeps and one for ranged creeps
- The Ancient, located at the far end of each base, protected by two Tier 4 towers

Each wave of lane creeps normally contains three melee creeps and one ranged creep. If both barracks in a single lane are destroyed, the opposing team begins spawning super

creeps in that lane. If all six barracks are destroyed, the opposing team spawns mega creeps in every lane—units that are significantly stronger and persist until the end of the match.

Neutral Creeps

Unlike lane creeps, neutral creeps are non-aligned, artificially controlled units that spawn in designated jungle areas across the map. They do not engage unless attacked and are defeated for gold and experience.

Gold Income (Economy)

Gold is earned from multiple sources:

- Killing enemy heroes
- Assisting in kills (teamfights)
- Killing lane creeps or neutral creeps
- Destroying towers, barracks, Roshan, and Tormentor
- Picking up Bounty Runes, which spawn every three minutes and provide teamwide gold

Gold allows players to purchase items, improving their impact in combat and overall match influence.

Experience (XP) System

Heroes earn experience points (XP) to level up and unlock new abilities. XP is gained through:

- Killing enemy heroes
- Assisting in kills
- Farming lane and neutral creeps
- Being near Roshan when it dies

A special Wisdom Rune begins spawning at minute 7 and then every seven minutes afterward. It grants XP to the player who collects it and their teammate with the lowest XP at that moment. Objectives such as towers and barracks do not grant XP.

Death Penalties

When a hero dies, they suffer a gold penalty and are removed from play for a set duration, known as respawn time. This downtime increases with hero level and can lead to temporary power imbalances, such as an opposing team pushing objectives while the dead hero is absent.

Teamfights

A teamfight is a large-scale skirmish involving multiple heroes from both teams. These clashes often occur near key map areas—such as Roshan, towers, or runes—and are decisive moments that can shift momentum. Winning a teamfight can result in large XP and gold rewards, as well as map control.

Teamfights typically involve coordinated ability usage, item activation, and strategic positioning.



Figure 1.1: Dota 2 game map image. Source: Author

Objective / location	Color
Tier 1 Towers	Orange
Tier 2 Towers	Purple
Tier 3 Towers + set of barracks	Light Blue
Tier 4 Towers + Ancient	Blue
Radiant jungle + Bounty rune spawn	Light Green
Dire jungle + Bounty rune spawn	Brown
Tormentor + Wisdom rune spawns	Black
Roshan spawns	Green
Active rune spawns	Yellow
Radiant and Dire bases	White and Grey home icons

Table 1.1: Map important locations. Source: Author

Focus on Professional Matches

This thesis concentrates exclusively on professional-level Dota 2 matches, where strategic depth and player skill are at their highest. These games minimize random or disruptive behaviour (common in casual play) and provide clean, structured data that is well-suited for predictive modelling.

Captain's Mode and Hero Draft

Professional games use a format called Captain's Mode, in which each team designates a captain responsible for selecting and banning heroes. Each team drafts five heroes and bans seven from a shared hero pool, following a fixed turn-based sequence. Once the draft is finalized, players choose from their team's selected heroes.

Some heroes are disabled in Captain's Mode, usually due to recent updates or balance changes, and the selection pool is curated directly by the developer (*Valve*).

Series Format

Professional matches are often played in series, depending on the tournament structure:

- Best-of-One (Bo1) – winner is decided in a single game
- Best-of-Three (Bo3) – first team to win two games wins the series
- Best-of-Five (Bo5) – first team to win three games wins the series

The format varies depending on the match stage, importance, and tournament design.

1.2 The Prediction Problem in Dota 2

As esports—especially games like Dota 2—have become more popular (*The Surprising History of Video Games and Esports*, 2024), interest in studying them has also grown. Researchers and fans have looked at Dota 2 from many angles, such as how players behave or how hero draft can affect the particular match. A common way to start this research is by using the large amount of data each match produces. One of the often-studied topics is predicting who will win a match based on in-game or pre-game information, though this is just one of many possible areas of focus.

Akhmedov and Phan (2021) aimed to predict final match outcomes using real-time data provided by the Game State Integrator (GSI) and various models such as neural networks and linear regression. This study is particularly interesting as the authors achieved solid accuracy using only in-game variables.

The connection between data analysis and Dota 2 extends beyond predicting the winner; outcomes of various game phases can also be forecasted. In their paper, Perry and Conley (2013) proposed a hero recommendation engine using machine learning techniques like Logistic Regression and K-Nearest Neighbors (KNN), intended to support Dota 2 players in making more informed draft decisions. While their system was not specifically designed

for professional use, such an approach could, in the authors' view, also benefit analysts, coaches, and professional players by enhancing strategic depth and gameplay preparation.

Another interesting concept was introduced by Ke et al. (2022), who created a formal definition of team fights and used this framework to predict match outcomes based solely on team fight events. Their results demonstrated that even this isolated component of gameplay can yield strong predictive performance.

Yuen, Thomson, and Don (2020) proposed a solution capable of detecting unexpected gameplay behaviour in Dota 2 matches using Logistic Regression, Random Forest and Multi-layer perceptron. Their approach focuses on preventing activities such as account boosting — where a player pays a more skilled individual to play on their account for financial compensation, artificially raising their matchmaking rating — and smurfing, where experienced players create or use lower-ranked accounts to dominate less skilled opponents mostly for personal enjoyment. Their best model achieved an accuracy of 95%.

Similarly, Yang, Qin, and Lei (2016) explored the problem of match winner prediction by leveraging both pre-game and in-game (real-time) data. They achieved a 93.73% accuracy by applying a time-specific logistic regression model, collecting real-time features in five-minute intervals.

2 Data Architecture overview

The data processing architecture in this thesis is designed to ensure transparency, traceability, and reusability across all stages of the machine learning pipeline.

It is built around a structured Extract–Transform–Load (ETL) flow and supported by a robust metadata system that tracks the entire lifecycle of the data, from initial extraction to model training. The architecture is divided into several logical components: the stage layer, target layer, and metadata layer.

This process is coded using Python. Example code is introduced in Appendix B.

Scripts for tables creation are introduced in Appendix A.

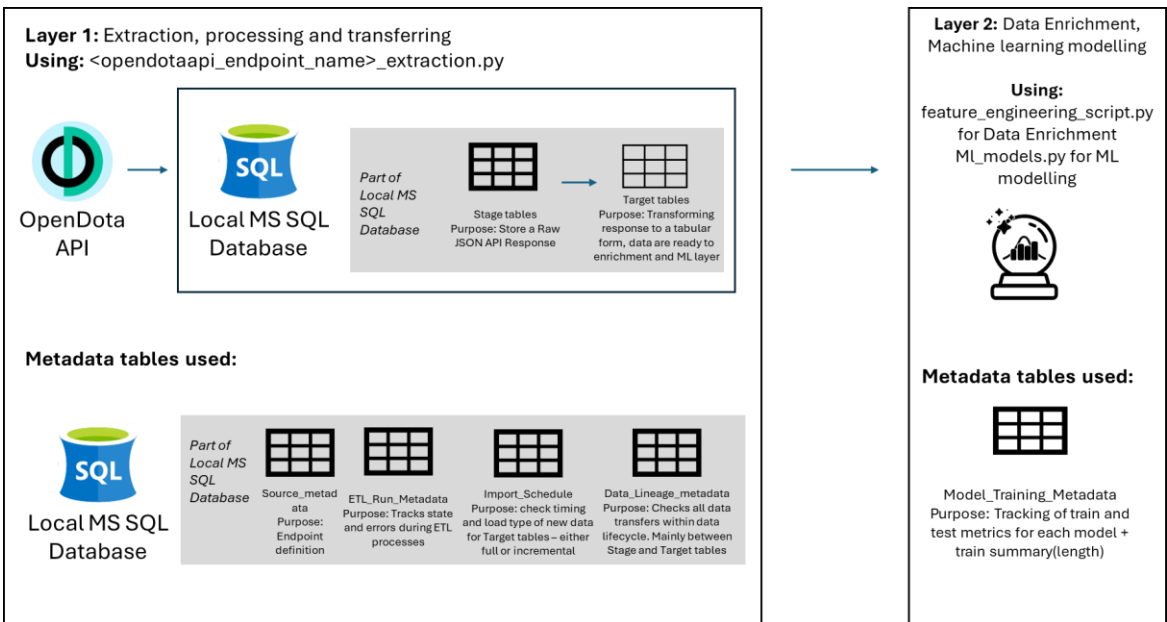


Figure 2.1: Data pipeline overview. Source: Author

Data Extraction and Source Metadata

The process begins with the extraction of data from designated OpenDota API endpoints. List of endpoints is introduced in Appendix C.

For each endpoint, extraction details are recorded in a centralized metadata table called metadata.Source_Metadata. This table includes the record ID, source name (OpenDota), the full API URL, a human-readable name of the endpoint, any parameters used (if applicable), and the exact extraction time (created_at). These metadata entries provide an auditable trail of where and when data was retrieved.

Stage Layer: Raw Data Holding Area

Once extracted from OpenDota, the data is stored in dedicated stage tables, each corresponding to a specific API endpoint. All stage tables are housed within a unified stage layer, which serves as a temporary holding zone for raw, unprocessed data. This layer maintains the original structure of the data (typically in JSON format).

Organizing raw data by endpoint into separate stage tables provides better modularity and simplifies troubleshooting in the event of schema changes or API issues.

Transformation and Target Layer

The raw data in the stage layer is then transformed into a structured, tabular format using predefined transformation scripts. Each endpoint has its own script that reshapes and cleans the data to fit a relational model suitable for analysis. The processed data is stored in corresponding target tables, all of which reside in a unified target layer.

The target layer represents the clean, analytics-ready zone of the pipeline. It provides a stable and standardized interface for downstream processes such as feature engineering and machine learning.

Data Lineage Tracking

To ensure the traceability of each data transformation, the system records all stage-to-target data transfers in the `metadata.Data_Lineage_Metadata` table. This includes the names of the source and target tables, timestamps of the operation, and a reference to the associated ETL run through the `run_id` field. This lineage metadata supports pipeline transparency and simplifies debugging and validation.

ETL Run Metadata

All major pipeline operations are logged in the `metadata.ETL_Run_Metadata` table. This includes entries for "Extraction to Stage", "Transform to Target", and "Feature Engineering", which involves further preparing the data for machine learning. This central logging mechanism ensures reproducibility and provides a clear operational history.

Model Training Metadata

The `metadata.Model_Training_Metadata` table stores detailed information about every model trained on the prepared data. It tracks the algorithm used, hyperparameters, duration of training, validation metrics, and the set of features involved. Each model entry is uniquely identified, allowing consistent performance tracking and experiment reproducibility.

Relation to the Model Lake Concept

This architecture reflects the principles of the "Model Lake" concept proposed by Garouani, Ravat, and Valles-Parlangeau (2024), where data is managed through distinct zones for ingestion, transformation, and modeling. The strong metadata and lineage tracking in this implementation function similarly to their governance layer, ensuring transparency and control across all stages of the data lifecycle.

3 Theoretical part

This section provides a comprehensive overview of the theoretical concepts underlying the machine learning methodology employed in this study.

It introduces fundamental evaluation metrics used to assess classification performance, as well as a selection of predictive models ranging from linear classifiers to ensemble-based approaches and neural networks. Emphasis is also placed on the methodological strategies for model validation, parameter optimization, and feature importance concepts.

3.1 Evaluation metrics

- **TP** represents number of true positive predictions
- **TN** represents number of true negative predictions
- **FP** represents number of false positive predictions
- **FN** represents number of false negative predictions

3.1.1 Accuracy:

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN}$$

is a ratio of correctly classified instances to the total number of instances. (Rainio et al., 2024)

3.2 Machine learning models introduction

Logistic Regression

Logistic Regression is a linear classification algorithm used for predicting binary outcomes. It estimates the probability that a given input belongs to the positive class using the logistic (sigmoid) function. Can be expressed as:

$$\log\left(\frac{P(y = 1 | x)}{1 - P(y = 1 | x)}\right) = \beta^T X + \mu$$

where:

- β is the vector of weights (coefficients)
- X is the vector of variables
- μ is the error term

XGBoost

XGBoost is an efficient implementation of gradient boosted decision trees. It builds an ensemble of decision trees sequentially, where each tree aims to minimize the errors made by the previous ones. XGBoost incorporates regularization and tree-specific optimizations to enhance both accuracy and speed (Chen & Guestrin, 2016). The prediction of the model is given by:

$$\hat{y} = \sum_{k=1}^K f_k(x), \quad f_k \in \mathcal{F}$$

where:

- $\mathcal{F}$ is the space of decision trees
- f_k is the k-th decision tree
- $\hat{y}$ is the predicted value

The regularized objective function minimized during the training is given by:

$$\mathcal{L} = \sum_{i=1}^n l(y_i, \hat{y}_i) + \sum_{k=1}^K \Omega(f_k)$$

where:

- $l(y_i, \hat{y}_i)$ is the loss function (logistic loss in case of classification)
- $\Omega(f_k)$ is the regularization term

Multilayer Perceptron (MLP)

The Multilayer Perceptron is a type of feedforward neural network composed of an input layer, one or more hidden layers, and an output layer. Each layer performs a linear transformation followed by a nonlinear activation function.

A feedforward neural network with one hidden layer is defined as:

$$h = f(W_1 X + b_1)$$

$$\hat{y} = g(W_2 h + b_2)$$

where:

- h is the output of hidden layer
- f is the activation function (for example, ReLU or tanh)
- g is the output activation function (Sigmoid for binary classification)
- W_1 is the weights matrix connecting input and hidden layer.
- W_2 is the weights matrix connecting hidden layer and the output.
- X is the vector of variables
- b_1 is the bias vector for the hidden layer

- b_2 is the bias vector for the output layer

3.3 Hyperparameter optimization

Hyperparameter optimization is the process of selecting the best configuration settings that control the behaviour and performance of machine learning models.

Hyperparameters are not learned during model training; instead, they are defined prior to training and influence how the model learns from data.

Now let's introduce list of hyperparameters for each model been used (optimized) during this thesis:

Logistic Regression hyperparameters used:

- `C` — Controls the strength of regularization. Smaller values apply stronger penalty to model complexity, which helps prevent overfitting.
- `l1_ratio` — Sets the mix between L1 (lasso) and L2 (ridge) regularization when using elastic net. A value of 0 means only L2 is used; a value of 1 means only L1.
- `penalty` — Specifies the type of regularization. 'elasticnet' combines both L1 and L2 penalties.
- `solver` — Defines the algorithm used to find the model's optimal weights.
- `class_weight` — Adjusts how much importance is given to each class, helping to handle imbalanced datasets.
- `max_iter` — Sets the maximum number of training iterations allowed for the solver to converge to a solution.

XGBoost hyperparameters used:

- `n_estimators` — Specifies the number of trees the model will build.
- `learning_rate` — Determines how much each tree contributes to the overall prediction.
- `max_depth` — Sets the maximum depth of each individual tree.
- `min_child_weight` — Defines the minimum sum of instance weights (i.e., number of samples) needed in a child node.
- `subsample` — Controls the fraction of training data used to build each tree.
- `colsample_bytree` — Sets the fraction of features (columns) to be randomly sampled for each tree.
- `reg_alpha` — Applies L1 regularization (lasso) on the weights.
- `reg_lambda` — Applies L2 regularization (ridge) on the weights.
- `eval_metric` — Specifies the evaluation metric used during training. 'logloss' is used for purposes of this thesis.

MLP hyperparameters used:

- `hidden_layer_sizes` — Defines the number of neurons in each hidden layer. For example, (256, 128, 64) represents a network with three hidden layers containing 256, 128, and 64 neurons respectively.
- `activation` — Specifies the activation function used in the hidden layers. Common choices include 'relu' (rectified linear unit) and 'tanh' (hyperbolic tangent). Those functions will be introduced later in a Practical Part.
- `alpha` — Sets the L2 regularization term to help control the magnitude of the model weights.
- `learning_rate_init` — Defines the initial learning rate used by the optimizer during weight updates.
- `early_stopping` — Enables stopping the training process if the model's performance on a validation set does not improve for a predefined number of iterations.
- `max_iter` — Specifies the maximum number of iterations allowed for training.
- `batch_size` — Sets the number of training samples used in each mini-batch for gradient computation.

Goal is to choose the best-performing ML model using hyperparameter optimization method combined with *k-fold cross validation*. (Soper, 2021)

3.3.1 Random Search

Random search is a method for hyperparameter optimization that involves randomly sampling combinations of hyperparameters from predefined distributions (Soper, 2021).

3.3.2 K-fold cross-validation

K-fold cross-validation is a model evaluation method used to estimate the performance of a machine learning algorithm on unseen data.

In *k-fold cross-validation*, the original dataset is split into k equal-sized subsets (folds). The model is trained k times, each time using $k - 1$ folds for training and the remaining one fold for validation.

After each iteration, evaluation metrics such as *Accuracy* or *F1 score* are calculated on the validation fold. This process continues until every fold has been used once as the validation set. Finally, the performance scores are averaged across all k iteration (Soper, 2021).

A visual representation of this process is shown in Figure 3.2.

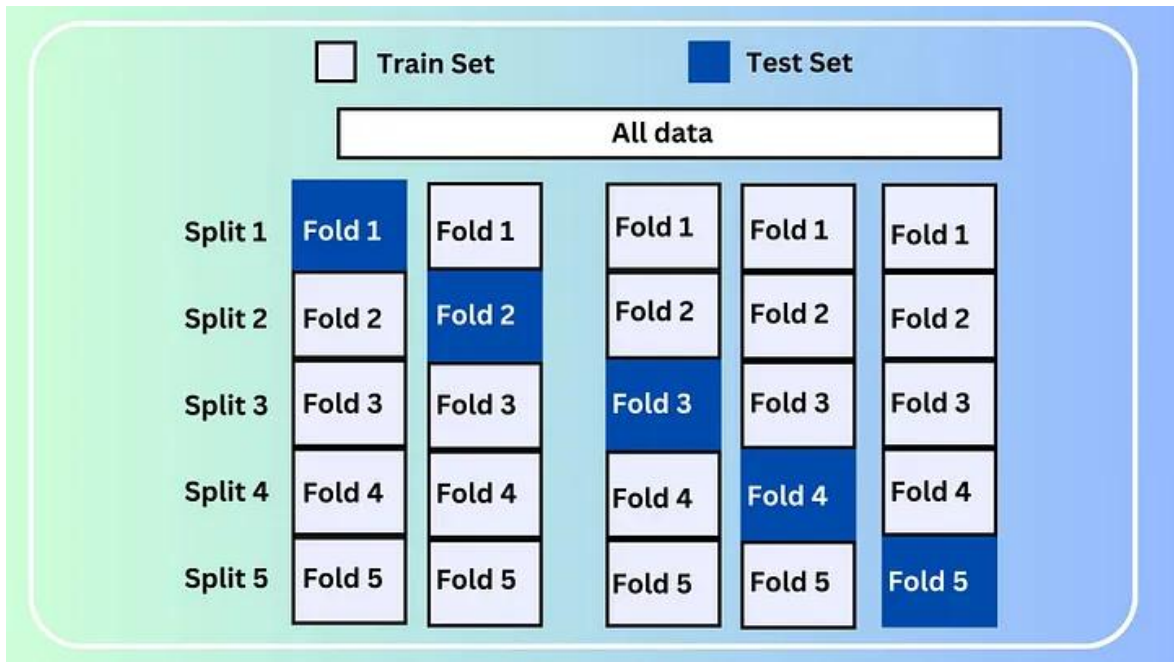


Figure 3.2: 5-fold cross-validation example. Source: (Nalawade, 2024)

3.4 Data transformation techniques overview

3.4.1 One-hot encoding

One-hot encoding is a data transformation technique that converts categorical (string) variable into a numerical format creating a binary vector for each category where “1” indicates a presence of that category and others are labelled as “0”. It can be particularly useful given that the machine learning models are based on Logistic Regression and MLP (Multi-Layer Perceptron) algorithms. (Samuels, 2024)

Label Encoding			One Hot Encoding			
Food Name	Categorical #	Calories				
Apple	1	95				
Chicken	2	231				
Broccoli	3	50				

→

Apple	Chicken	Broccoli	Calories
1	0	0	95
0	1	0	231
0	0	1	50

Figure 3.3: One-hot encoding example. Source: (Samuels, 2024)

3.4.2 Binning

Discretization converts continuous features into discrete intervals, and binning is one way to perform this transformation.

Two types of binning will be used:

- Custom – for example, splitting early game phase (10 minutes) into 2 intervals: 0-5, 5-10.
- Quantile binning – the continuous variable is split based on percentiles, typically into quartiles.

3.5 T-test

A one-sample t-test is a statistical method used to determine whether the mean of a single sample significantly differs from a known or hypothesized population mean, particularly when population parameters are unknown (Kim, 2015).

Mathematically, t-statistic is defined as:

$$t = \frac{\bar{x} - \mu_0}{\frac{s}{\sqrt{n}}}$$

where:

- $\bar{x}$ is the sample mean
- μ_0 is the hypothesized population mean
- s is the sample standard deviation
- n is the sample size

The resulting t-statistic follows a Student's t-distribution with $n-1$ degrees of freedom. Based on the t-distribution, a *p-value* is calculated to assess the significance of the result.

3.6 Feature importance

Feature importance refers to quantifying how much each input variable contributes to a model's prediction. Understanding feature importance helps interpret model decisions and select informative features. Guyon and Elisseeff (2003) offer a foundational overview of the topic, categorizing methods into filter, wrapper, and embedded approaches. This study applies model-specific techniques to extract feature importance from different model families — logistic regression, gradient boosting, and neural networks.

3.6.1 SHAP values & beeswarm plot

For tree-based models like XGBoost, SHAP (SHapley Additive exPlanations) values provide a theoretically grounded approach to feature attribution.

Based on cooperative game theory, SHAP assigns each feature a value representing its contribution to the prediction, ensuring both consistency and local accuracy (Lundberg & Lee, 2017).

Averaging the absolute Shapley values across all samples yields a global measure of feature importance. This method is particularly useful in capturing non-linear and interaction effects present in ensemble models.

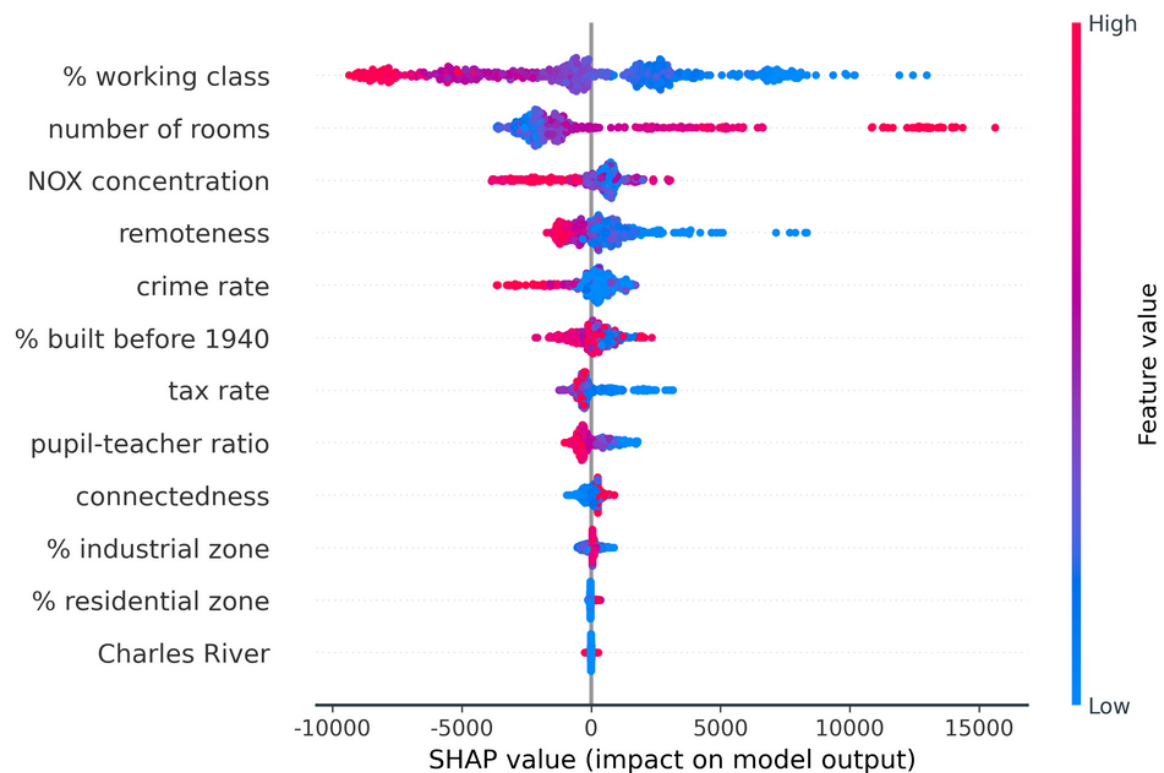


Figure 3.3 Shapley values beeswarm plot example. Source: (Cooper, 2024)

In this example, the Shapley values beeswarm plot is used to show how each feature influenced the model's predicted house prices across the dataset. The x-axis represents the Shapley value — that is, how much a feature increased or decreased the predicted price for a given observation. Each dot represents one prediction.

Features are ranked from top to bottom by overall importance, meaning that the *% working class* feature appears to be the most influential.

Narrow spreads centred around zero indicate that a feature has low or inconsistent impact — the model struggles to differentiate its effect.

Wider spreads along the x-axis suggest that the feature has a stronger influence on the model's predictions.

Red dots represent high values of the feature, while blue dots indicate low values.

In this example, a high percentage of working-class population is associated with lower predicted house prices, while a greater number of rooms tends to increase the predicted price.

3.6.2 Coefficient-based importance in Logistic Regression

In linear models such as logistic regression, feature importance is often derived from the absolute magnitude of the model's coefficients. Each coefficient represents the weight applied to a feature in the log-odds function. Features with larger absolute values have a greater effect on the outcome, making this a direct and interpretable method of assessing variable importance (Guyon & Elisseeff, 2003). Regularization techniques like elastic net can also help shrink less important coefficients toward zero, making interpretation more robust in high-dimensional settings.

3.6.3 Permutation-based importance in Neural Networks

For neural networks such as multilayer perceptron (MLP), feature importance can be measured using permutation importance. This model-agnostic method evaluates the decrease in model accuracy when the values of a single feature are randomly shuffled. The greater the performance drop, the more important the feature is to the model's predictions. Altmann et al. (2010) improved this approach by correcting for biases inherent in earlier implementations, making it more reliable in practical settings.

4 Practical part

This chapter is dedicated to exploring the factors that influence match outcomes through data enrichment and machine learning processes. Special attention is given to model selection, hyperparameter optimization, and improving the robustness of the models when applied to real-world data scenarios.

4.1 Dataset overview

Let’s begin by familiarizing ourselves with the dataset’s features (variables). The dataset first 10 rows preview can be found in Appendix D. The full list of features can be found in Appendix E.

The initial dataset consists of 23,048 matches (rows) and 38 features (columns). All matches were played between May 22, 2024 (the release of patch 7.36)¹ and February 19, 2025 (the release of patch 7.38).²

In-game features are taken from the first 10 minutes, while pre-game features happen before the match and are not time-limited. “Feature Group” is shown just to help the reader understand the type of each feature.

Column name	Column description	Feature Group
match_id	The ID number of the match assigned by Valve	In-game
radiant_win	Boolean indicating whether Radiant won the match. 1 – Radiant won, 0 – Dire won. Target variable	In-game
networth_diff_10min	Difference of gold amount between sides. Positive value indicates Radiant side advantage, negative value indicates Dire side advantage	In-game
first_pick	Indicates which team had the first hero pick during the draft stage	Pre-game

Table 4.1: Main dataset brief overview. Source: Author

¹ Valve. (2024). *Dota 2 Patch Notes - 7.36*.
² Valve. (2025). *Dota 2 Patch Notes – 7.38*.

4.2 Data transformations

This section is dedicated to data transformations so that the reader can follow the entire preprocessing pipeline that led to the final predictions and potentially replicate the solution for their own purposes.

4.2.1 Elo ratings

The first transformation performed on the initial dataset was the binning (a combination of quartile-based and custom binning) and one-hot encoding of the *radiant_elo* and *dire_elo* variables, which represent the Elo scores assigned to the squads playing as the Radiant or Dire side in each match.

The known Elo scores were used to calculate the 25th, 50th, and 75th percentiles. A custom binning function then assigned each Elo score to one of the following categories:

- Bottom 25%
- 25–50%
- 50–75%
- Top 25%
- Unknown (for missing values)

Special attention was given to the Unknown category, as most of the matches do not include Elo information. Specifically, 17,340 values were missing in the *radiant_elo* column, and 17,273 in the *dire_elo* column. The reason for this missingness is that many of the teams represented in the dataset are not part of the Elo scoreboard, making it impossible to assign them a rating.

These missing values can be classified as *Missing Not at Random (MNAR)* (Nahhas, 2025b), as the probability of missingness is dependent on unobserved characteristics—namely, a team's lack of competitive history.

Finally, one-hot encoding was applied to the two binned columns. In both cases, the 25–50% category was dropped and treated as the reference level.

Original variables were then dropped to prevent redundancy.

4.2.2 Tower destruction timings

Tower destruction times (*first_mid_tower_time*, *first_safelane_tower_time*, *first_offlane_tower_time*) were binned using *custom binning* based on author preference to explore impact of early game tower destruction factors. Single category is reflecting minutes interval.

The resulting categories were:

- 0–5
- 5-10
- Not destroyed in 10 min (for missing values)

Missing values were treated as a distinct "Not destroyed in 10 minutes" category, representing towers that were either destroyed after the 10-minute mark or not destroyed at all during the match. This missingness is classified as **Missing Not at Random** (MNAR), as the absence of data is inherently linked to an unobserved but meaningful game event — specifically, the tower’s survival beyond the analysis window (Nahhas, 2025b).

4.2.3 Rest of string-type variables

The remaining string-type variables (such as *drums_presence*) were transformed using One-Hot encoding with the `drop_first=True` argument. See Appendix E for a list of dropped (referenced) categories.

4.2.4 *first_blood_time* transformation

Log transformation was applied to *first_blood_time* feature because variable distribution graph is severely right skewed.

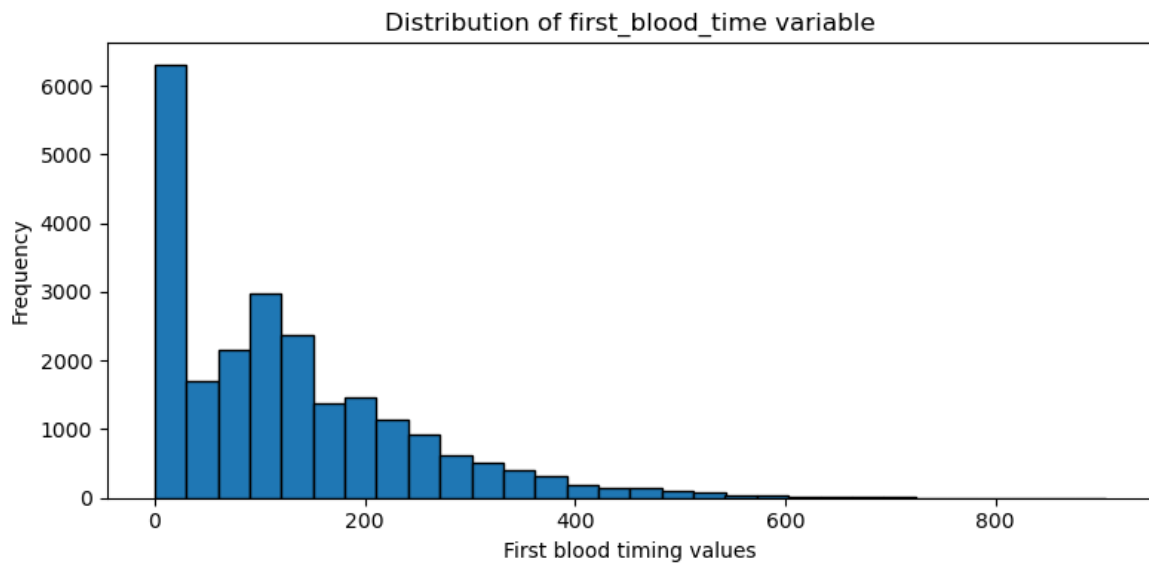


Figure 4.1: Distribution of original *first_blood_time* variable. Source: Author

4.3 Features exploring & model creation

After all transformations are applied and the data engineering process is completed, the distribution of the target variable — “*radiant_win*” — and the dimensionality of the dataset are examined.

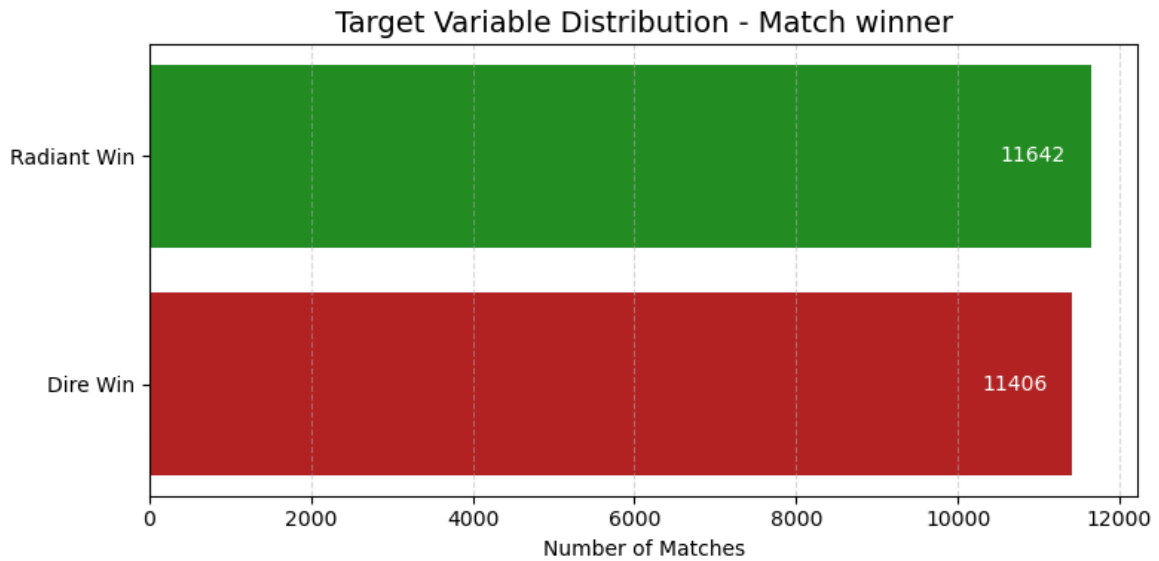


Figure 4.: Target variable distribution – “radiant_win”. Source: Author

After converting string variables into categorical form using one-hot encoding, the dataset contains 23,048 matches: Radiant won 11,642 times and Dire won 11,406 times. From the perspective of the target variable, this can be considered a balanced dataset. The resulting number of variables is 53.

As a starting point, it is worth checking whether teams playing on the Radiant side tend to win more often. This simple test can provide initial insight into whether side selection has any measurable impact on match outcomes.

Sample size (n)	Observed Radiant win rate	Null hypothesis (H0)	Alternative hypothesis (H1)	t-statistic	One-sided p-value
23,048	50.512 %	$\mu = 0.500$	$\mu > 0.500$	1.555	0.0600

Table 4.2: Test of Radiant side advantage in match outcomes. Source: Author

To determine whether playing on the Radiant side offers a statistically significant advantage in terms of win rate, a one-sample t-test was conducted. The binary outcome variable `radiant_win` (1 = Radiant win, 0 = Radiant loss) was tested against the null hypothesis that the population mean win rate is 0.5, representing no side advantage.

The alternative hypothesis was that the Radiant side has a win rate greater than 0.5 (i.e., Radiant is more likely to win).

The observed win rate across 23,048 matches was 50.512%, with a computed t-statistic of 1.555. The corresponding one-sided p-value was 0.0600, which is above the conventional 0.05 significance level.

Thus, the result is not statistically significant at the 5% level. The null hypothesis is not rejected, indicating that there is no significant Radiant side advantage at this level. The probability of winning is considered equal across both sides.

Let's now look at the results of tower destruction timings binning:

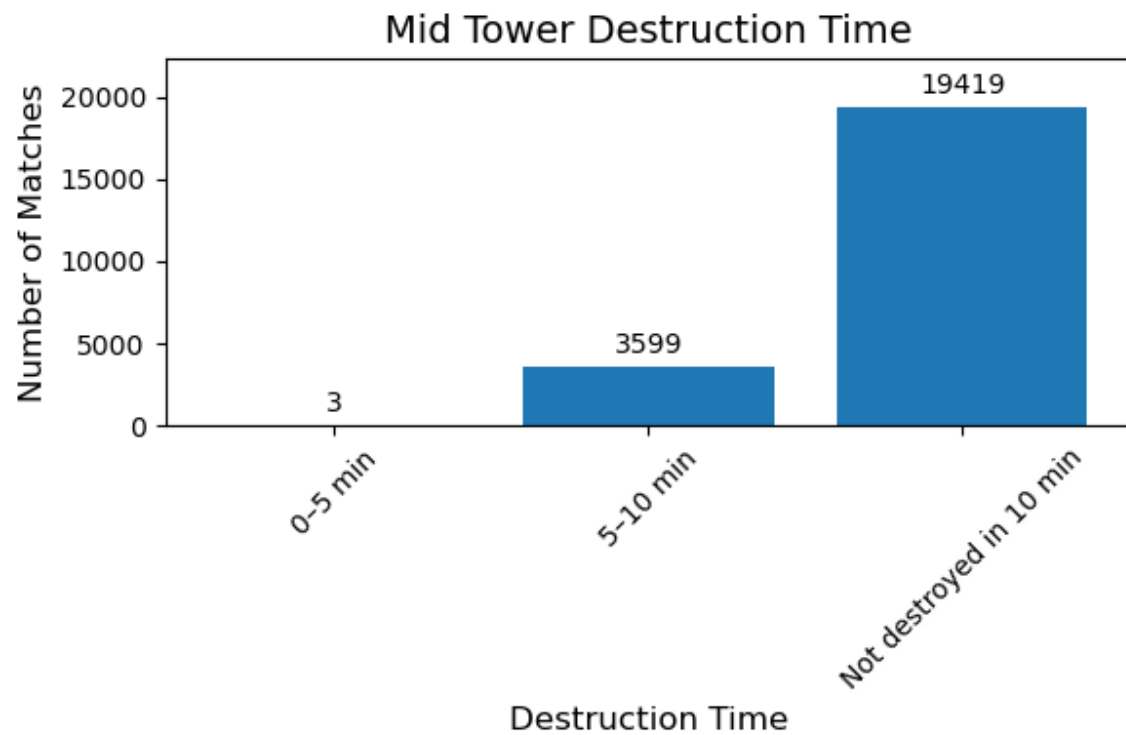


Figure 4.1: First mid lane tower destruction timings binning. Source: Author

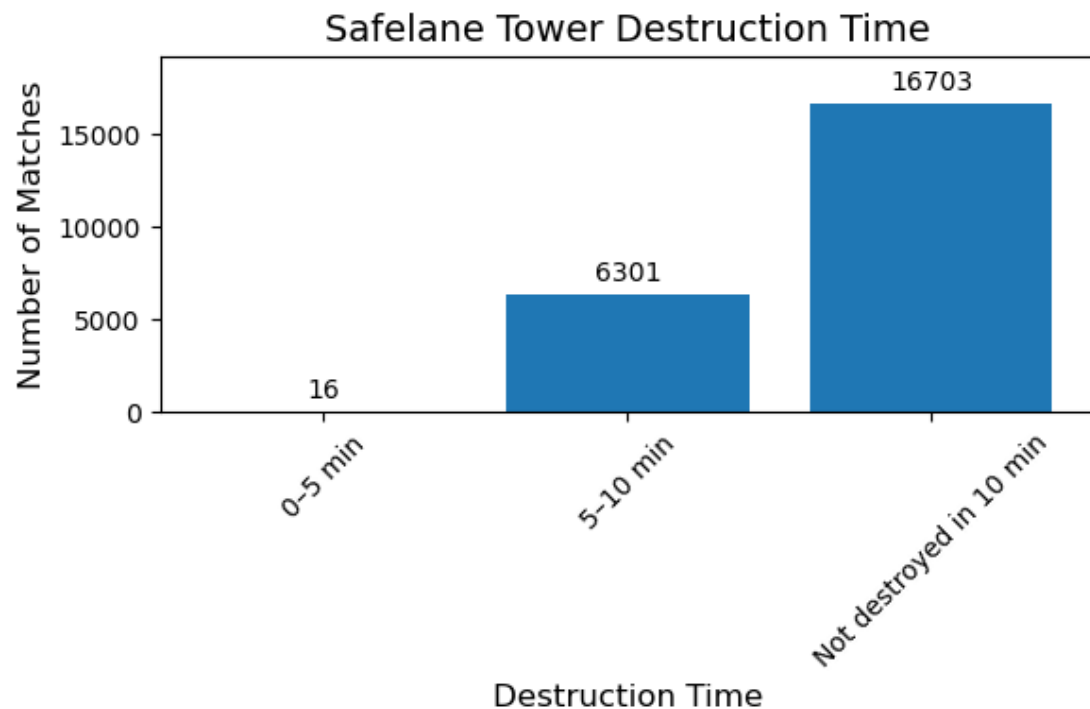


Figure 4.2: First safe lane tower destruction timings binning. Source: Author

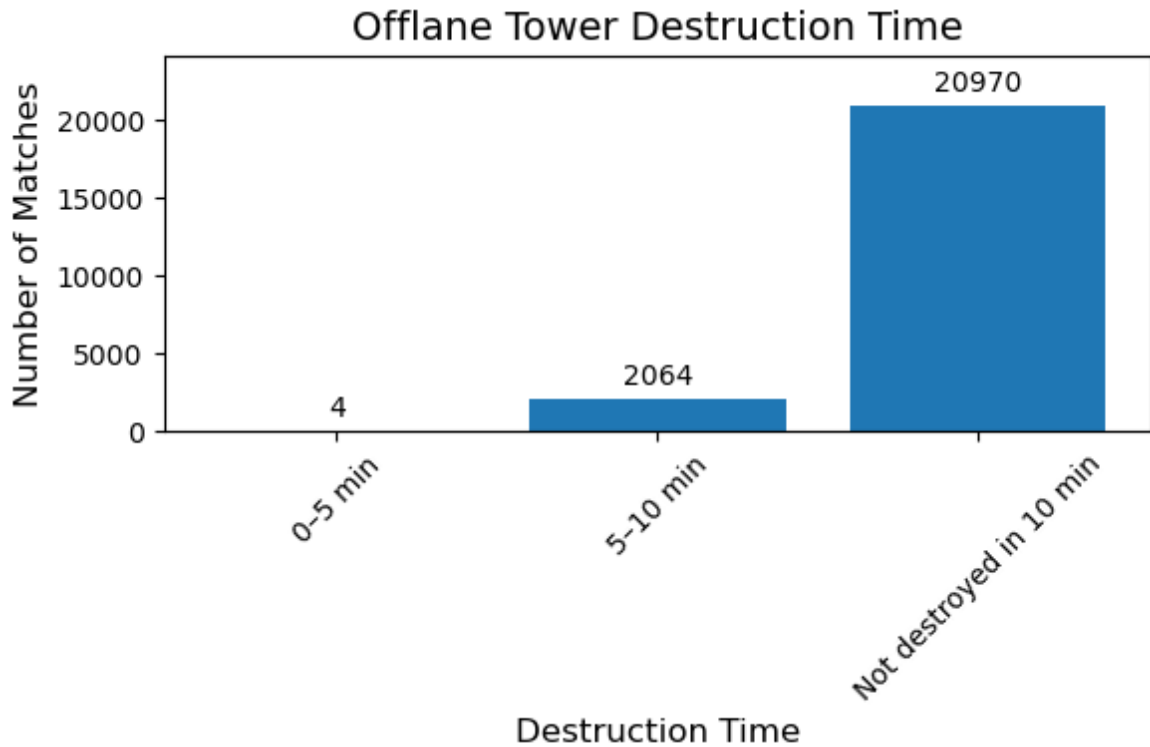


Figure 4.3: First off lane tower destruction timings binning. Source: Author

Based on the three visualizations, the safe lane Tier 1 tower emerges as the most frequently destroyed tower within the first 10 minutes of the game. This observation aligns with strategic expectations: the enemy off lane duo is typically stronger than the opposing safe lane duo during the first 10 minutes of the game. Destroying the safe lane Tier 1 tower yields significant strategic benefits, such as access to two segments of the enemy jungle and control over the nearby bounty rune spawn.

The mid lane Tier 1 tower ranks as the second most frequently destroyed. While this tower is often considered the most strategically important due to the space it controls — including both active rune spawns, two jungle entrances, and fast rotational access to either side lane — it is typically better defended. As a result, its destruction within the first 10 minutes is less common despite its tactical importance.

In contrast, the off lane Tier 1 tower is the least likely to be destroyed within the 10-minute window. This is because the safe lane duo, which generally occupies this lane, tends to be weaker during the early game compared to the opposing off lane duo. Furthermore, the strategic space gained by destroying this tower is relatively limited.

Overall, most tower destructions occur between the 5–10 minute interval or later. Tower destructions observed in the 0–5 minute window are rare and often represent exceptional cases, such as extreme early-game dominance by one team.

Train-Test Splitting

A stratified train-test split was performed, using 80% of the data for training and 20% for testing. Stratification was based on the target variable *radiant_win* to maintain the original

class balance across both sets. The same indices were used for both dataset versions to ensure fair and consistent comparison of model performance across different preprocessing strategies.

Dataset preparation for model training

This thesis employs three base models: XGBoost, Logistic Regression, and Multi-Layer Perceptron (MLP). These models are later combined into a stacked ensemble using a Random Forest as the meta-model.

To accommodate the specific characteristics of these models, the transformed dataset introduced earlier in this chapter was divided into two versions: one used for XGBoost and MLP, and another specifically adapted for Logistic Regression.

The motivation for this division was to address the presence of outliers in two continuous features: `xp_diff` and `networth_diff_10min`.

The dataset used for XGBoost & MLP remained unchanged. Tree-based models such as XGBoost are generally considered robust to outliers due to their internal structure and built-in regularization mechanisms (Chen & Guestrin, 2016).

In contrast, the dataset prepared for Logistic Regression underwent additional transformation. The `xp_diff` and `networth_diff_10min` variables were discretized into categorical bins, reflecting both the direction and magnitude of the early-game advantage. The resulting categories include:

- draw (no difference),
- moderate, high, and extreme advantage for either Radiant or Dire.

These new categorical features were then one-hot encoded, and the original continuous columns were removed from the Logistic Regression dataset. According to the scikit-learn documentation, discretizing continuous features can enhance the predictive power of linear models by enabling them to capture non-linear relationships that would otherwise be difficult to model directly (Using KBinsDiscretizer to Discretize Continuous Features, 2025).

Machine learning models used for evaluation

To model the probability of a Radiant victory, a logistic regression framework is applied.

This model estimates the log-odds of the Radiant team winning based on a set of explanatory variables and takes the following form:

$$\log \left(\frac{p(\text{radiant_win})}{1 - p(\text{radiant_win})} \right) = \alpha + X\beta + \mu$$

X - vector of variables described earlier + binned `xp` and `networth` differences,

β - vector of weights (coefficients)

μ - error term

In addition to logistic regression, Extreme Gradient Boosting (XGBoost) is employed — a powerful ensemble learning technique based on gradient-boosted decision trees. XGBoost is well-suited for structured data and has demonstrated strong performance in classification tasks (Chen & Guestrin, 2016).

The model combines the outputs of multiple regression trees, each correcting the residuals of the previous ones in a stage-wise manner.

Formally, the predicted log-odds of a Radiant win are expressed as the sum of predictions from individual trees:

$$\log \left(\frac{p(\text{radiant_win})}{1 - p(\text{radiant_win})} \right) = \sum_{i=1}^n f_i(X)$$

X – vector of variables described earlier; n – number of trees

A Multilayer Perceptron (MLP) model — a type of fully connected feedforward neural network — is also employed as part of the classification experiments. All MLP models in this study use a common architecture consisting of three hidden layers with 256, 128, and 64 nodes, respectively. This structure is applied consistently as a baseline across all configurations.

The default activation function for the hidden layers is ReLU (Rectified Linear Unit), defined as:

$$\text{ReLU}(x) = \max(0, x)$$

It outputs the input directly if it is positive; otherwise, it returns zero. ReLU is computationally efficient and helps mitigate the vanishing gradient problem, which often hinders the training of deep networks.

By allowing only positive values to pass through, ReLU introduces sparsity into the network, which can improve generalization and reduce overfitting.

Second activation function that will be employed in sections including hyperparameters optimization is a tanh (Hyperbolic Tangent) function, which is defined as:

$$\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$$

It maps input values to the range $[-1, 1]$ and is zero-centered, which can make optimization via gradient descent more stable in some cases.

Unlike ReLU, tanh outputs both positive and negative values, allowing for symmetric learning behavior.

However, it is more susceptible to the vanishing gradient problem, especially when inputs fall into the flat regions of the curve where gradients become very small (Rasamoelina, Adjailia, & Sincak, 2020).

Initial models evaluation

This section presents baseline performance of three machine learning models trained on the initial dataset without any feature reduction techniques such as PCA or pruning. Similarly, hyperparameters optimization was not done on this version of models.

The models used include Logistic Regression, Multi-layer Perceptron (MLP), and XGBoost.

Each model was trained on a different version of the dataset:

- Logistic Regression was trained on a binarized (binned) feature set that includes categorized XP and network differences.
- MLP and XGBoost used the raw dataset, preserving continuous feature values.

The models and their core configurations were as follows:

Model	Initial Hyperparameters
Logistic Regression	C=1.0, solver='saga', penalty='elasticnet', l1_ratio=0.5, max_iter=5000, class_weight='balanced'
MLP	hidden_layer_sizes=(256,128,64), alpha=1e-4, learning_rate_init=1e-3, early_stopping=True, max_iter=1000, activation='relu', batch_size=64
XGBoost	n_estimators=150, max_depth=4, learning_rate=0.15, subsample=0.8, colsample_bytree=0.8, eval_metric='logloss', min_child_weight=3

Table 4.3: Initial models setting. Source: Author

Below is the resulting performance based on *accuracy*:

Model	Accuracy
Logistic Regression	65.7%
MLP	65.0%
XGBoost	65.3%

Table 4.4: Initial models performance. Source: Author

Based on the initial model setup, all three models — Logistic Regression, MLP, and XGBoost — achieved similar performance, with accuracies ranging narrowly between 65.0% and 65.7%.

Logistic Regression slightly outperformed the others, suggesting that even a linear model captures meaningful structure in the data. However, the differences are minimal at this stage, indicating that further tuning or feature engineering may be necessary to unlock the full potential of more complex models like MLP and XGBoost.

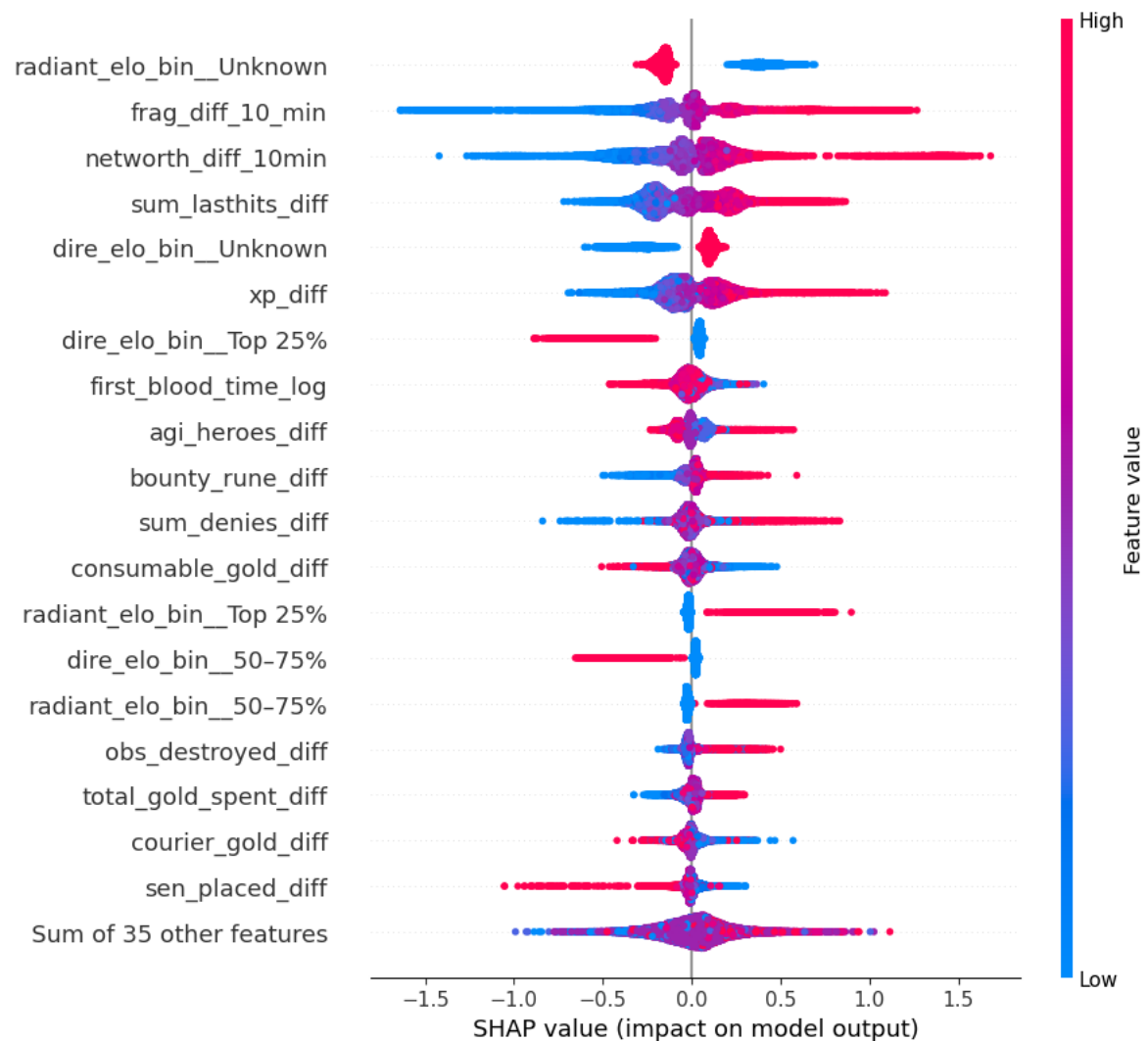


Figure 4.4: Shapley values beeswarm graph – initial XGBoost setup. Source: Author

The Shapley values summary plot illustrates how the XGBoost model differentiates between impactful and less relevant features when predicting Radiant victory. The model clearly assigns high importance to `radiant_elo_bin_Unknown`, `frag_diff_10_min`, and `networth_diff_10min`, indicating that it strongly responds to both team composition (in terms of skill tiers) and early-game combat and economic momentum.

The wide horizontal spread of Shapley values for these top features shows that the model is sensitive to both low and high values — for instance, large positive `frag_diff_10_min` values (i.e., Radiant getting early kills) push predictions toward a Radiant win, while negative values push it toward a Dire win. This reflects the model’s ability to capture nuanced game dynamics early in the match.

Additionally, variables like *sum_lasthits_diff*, *xp_diff*, and *bounty_rune_diff* show that the model recognizes smaller but still meaningful advantages in collecting gold and gaining experience.

The presence of several *elo_bin* features (e.g., *radiant_elo_bin__Top 25%*, *dire_elo_bin__50–75%*) indicates the model accounts for relative player strength, further validating that both pre-game skill indicators and in-game actions are jointly influencing its decisions.

Feature pruning – Logistic Regression

In the context of Logistic Regression, model coefficients represent the influence of each individual feature on the predicted log-odds of the target variable—in this case, the probability of a Radiant victory.

A higher absolute value of the coefficient indicates a stronger influence on the model's decision, while values close to zero suggest that the variable contributes little to the prediction process. To simplify the model and avoid overfitting, the 10 features with the lowest coefficient magnitudes were removed from the training dataset.

Interestingly, a similar set of weak predictors emerged across all models (Logistic Regression, MLP, and XGBoost), reinforcing their low predictive value regardless of algorithm. This convergence validates their exclusion and supports the stability of the feature selection process.

A few examples of pruned features and why they might be less informative:

- *lotus_picks_diff*: Fact of picking Greater Healing lotus is not so important generally because effect of its usage is immediate and affects only a few seconds of the game.
- *xp_diff_cat__dire_extreme*: While experience differences may generally be useful, this specific bin might be too narrow or rare to impact predictions meaningfully. (Applicable only for Logistic regression)
- *obs_placed_diff*: The total difference in placed observer wards may not reflect key strategic placements that matter more than quantity, especially within the first 10 minutes.
- *drums_presence__None*: This binary feature flags when neither team purchased Drums. This occurs in most matches; it provides little variance and becomes less discriminative.

To further improve model performance and reduce unnecessary dimensionality, feature pruning was applied to MLP and XGBoost models as well. Although the rationale remained consistent — removing features with the lowest contribution to model prediction — the methods used to assess feature importance were specific to each model type.

Feature pruning – MLP

For the MLP model, permutation importance was calculated using the accuracy metric as an evaluation measure. This method estimates how much the model's performance drops when the values of a given feature are randomly permuted.

Top 5 features with the lowest contribution been excluded from the MLP dataset. Among the least relevant features were *courier_gold_diff*, *agi_heroes_diff*, and *sen_placed_diff*, which either contributed very little or introduced noise into the model.

Feature pruning – XGBoost

In the case of XGBoost, the average absolute Shapley value of each feature was calculated across the dataset. Similarly, top 10 features with the lowest contribution, such as *drums_presence_None*, *lotus_picks_diff* and *vladmir_presence__None*, were removed from the final dataset prepared for XGBoost training.

Logistic Regression performance after feature pruning

The following table presents the evaluation results of the Logistic Regression model trained on the pruned dataset (i.e., after removing 10 least important features based on coefficient magnitude).

No hyperparameter optimization was applied in this step — model settings remained the same as the initial baseline.

Model	Accuracy
Logistic Regression (pruned only)	65.6%

Table 4.5: Logistic Regression performance after feature pruning. Source: Author

Although performance slightly decreased in comparison to the full-feature baseline (65.7%), the reduction was marginal. This suggests that the removed features had minimal predictive value and confirms the reliability of the pruning strategy.

MLP and XGBoost performance after feature pruning

To evaluate the effect of model-specific feature pruning, both MLP and XGBoost were retrained on reduced versions of their input datasets.

- For MLP, the 5 least impactful features were removed based on permutation importance.
- For XGBoost, the 10 least important features were removed using Shapley values.

This process aimed to reduce input dimensionality, potentially improve generalization, and remove irrelevant or redundant variables. The following table presents the updated performance metrics for both models on their pruned datasets:

Model	Accuracy
MLP (Pruned)	64.9%
XGBoost (Pruned)	65.4%

Table 4.6: MLP and XGBoost performance after feature pruning. Source: Author

- **MLP:** The pruning step slightly reduced the performance of MLP (approx. -0.1 percentage point of accuracy), suggesting that some of the dropped features might have had weak but non-negligible contributions. This is not uncommon for neural networks, which can extract complex patterns from a wide feature space.
- **XGBoost:** In contrast, XGBoost slightly benefited from pruning, improving both metrics. This aligns with expectations, as tree-based models can become noisy or less efficient when fed with too many weak predictors.

The following graph presents the Shapley values summary plot for the pruned XGBoost model.

It highlights the most influential features contributing to the model’s predictions, helping to understand which early-game factors most strongly affect the probability of victory in professional matches.

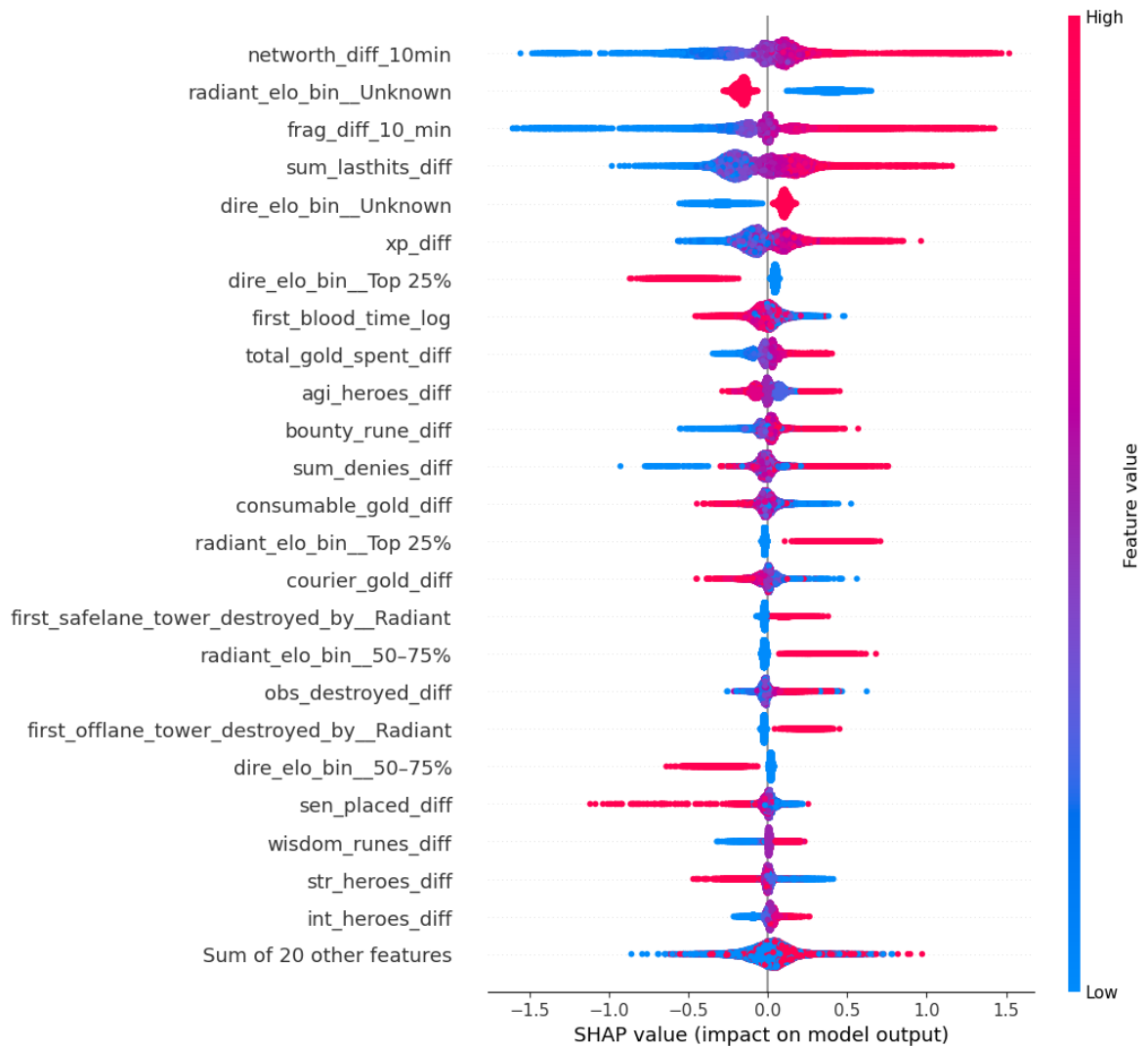


Figure 4.4: Most influential features in the pruned XGBoost model, sorted by their impact on the prediction. Source: Author

After feature pruning, the net worth difference between teams increased in importance, while the kill difference and the one-hot-encoded variable *radiant_elo_bin__Unknown* decreased in relevance. Overall, the relative importance of features remains largely unchanged.

Logistic Regression – feature pruning + hyperparameters optimization

In this step, the Logistic Regression model was refined using two key approaches:

1. Feature pruning – the 10 least important variables (based on coefficient magnitude) were removed.
2. Hyperparameter tuning – a Random Search was applied over an extended space of regularization and sparsity settings.

After pruning, the model was trained on the scaled dataset using the best hyperparameters found via 5-fold cross-validated randomized search.

Hyperparameters space for Logistic Regression is shown below:

Hyperparameter	Search Range
C	[0.0001, 0.001, 0.01, 0.1, 1, 10, 100]
l1_ratio	[0.0, 0.1, 0.25, 0.5, 0.75, 0.9, 1.0]
Penalty	'elasticnet'
Solver	'saga'
Class Weight	'balanced'
Max Iterations	5000

Table 4.7: Logistic Regression hyperparameters space. Source: Author

Best hyperparameters configuration derived from Random Search is:

Hyperparameter	Value
C	0.01
l1_ratio	0.1
Penalty	'elasticnet'
Solver	'saga'
Class Weight	'balanced'
Max Iterations	5000

Table 4.8: Logistic Regression best configuration derived from the Random Search. Source: Author

Optimized Logistic Regression shows the following performance:

Metric	Value
Accuracy	65.7%

Table 4.9: Logistic Regression best configuration derived from the Random Search results. Source: Author

Those results are same as initial Logistic Regression model is shown before.

XGBoost and MLP – feature pruning + hyperparameters optimization

In this step, hyperparameters optimization and corresponding results will be shown for XGBoost and MLP models. Hyperparameters optimization was proceed using Random Search.

Hyperparameter	Search Range
n_estimators	[100, 200, 300, 500]
learning_rate	[0.01, 0.05, 0.1, 0.2]
max_depth	[3, 4, 5, 6, 8]
min_child_weight	[1, 3, 5, 7]
subsample	[0.6, 0.8, 1.0]
colsample_bytree	[0.6, 0.8, 1.0]
reg_alpha	[0, 0.1, 1.0]
reg_lambda	[1.0, 5.0, 10.0]

Table 4.10: XGBoost hyperparameters space. Source: Author

Best hyperparameters configuration for XGBoost is shown below:

Hyperparameter	Value
n_estimators	100
learning_rate	0.05
max_depth	4
min_child_weight	3
subsample	1.0
colsample_bytree	0.8
reg_lambda	5.0
reg_alpha	1.0

Table 4.11: XGBoost best configuration derived from Random Search. Source: Author

Optimized XGBoost results:

Metric	Value
Accuracy	65.6%

Table 4.12: XGBoost hyperparameter-optimized performance. Source: Author

Next MLP was optimized and hyperparameters space is shown in a table below:

Hyperparameter	Search Range
Hidden Layer Sizes	(64,), (128, 64), (256, 128, 64)
Activation	relu, tanh
Alpha	0.0001, 0.001, 0.01
Learning Rate Init	0.001, 0.01

Table 4.13: MLP hyperparameters space. Source: Author

Below is the best hyperparameters configuration derived from Random Search for MLP:

Hyperparameter	Value
Activation	tanh
Alpha	0.01
Hidden Layer Sizes	(256, 128, 64)
Learning Rate Init	0.001

Table 4.14: MLP best configuration derived from Random Search. Source: Author

This optimization gave us the following result:

Metric	Value
Accuracy	64.4%

Table 4.15: MLP hyperparameter-optimized performance. Source: Author

Among the pruned and tuned models, XGBoost showed a slight improvement in accuracy compared to its baseline version.

Logistic Regression, despite both pruning and optimization, retained the same performance as its initial configuration, indicating that the dropped features were redundant, and that the model's capacity was already saturated.

MLP delivered the lowest accuracy among all three models, suggesting that either the architecture requires further tuning or that it struggles to generalize effectively given the current feature space.

Stacked model introduction

As the final phase of the exploration, a stacked model is introduced. The concept of stacking, also known as stacked generalization, is an ensemble learning technique introduced by Wolpert (1992).

It involves training multiple base learners independently—such as the models evaluated in this chapter (Logistic Regression, MLP, and XGBoost)—and then using their predictions as input features for a higher-level meta-learner.

This meta-model, which can be any machine learning algorithm (e.g., Logistic Regression or Random Forest), learns how to optimally combine the outputs of the base models.

This strategy aims to enhance overall predictive performance by leveraging the individual strengths of different algorithms while mitigating their respective weaknesses. In the context of this thesis, stacking serves as the final evaluation step, combining the best-performing base models into a single, potentially more robust ensemble.

Stacked model creation

The stacked model was constructed using the best-performing versions of each individual model.

Specifically, the base learners included:

- The optimized XGBoost model (pruned and hyperparameter-tuned)
- The optimized Logistic Regression model (after feature pruning and regularization)
- The baseline Multilayer Perceptron (MLP)

This combination was designed to leverage the complementary strengths of each model type—nonlinear tree-based learning, linear interpretability, and deep learning capacity.

In the context of the experiment, the pruned and tuned Logistic Regression model is considered the best single linear model due to its reduced complexity, improved generalizability, and identical predictive performance compared to the baseline.

Although no accuracy gain was observed after pruning and hyperparameter tuning, the model maintained its performance with fewer features and applied regularization, indicating that redundant inputs were successfully removed.

Random Forest model was chosen as meta-learner model. Algorithm is based on principle of bagging. It constructs multiple decision trees and combines their output, final decision is made by majority voting across all trees (in classification tasks, such as the case).

The prediction function for classification can be expressed as:

$$\hat{y} = \text{majority_vote}\{T_1(x), T_2(x), \dots, T_N(x)\}$$

where:

- $T_i(x)$ is the prediction of the i -th decision tree
- N is the total number of trees in the forest
- $\hat{y}$ is the final prediction

When building a stacking model using the *StackingClassifier* function from the *sklearn.ensemble* module in Python, setting *passthrough=False* ensures that the meta-learner uses only the predicted outputs of the base models as input. The original features are excluded, allowing the meta-learner to concentrate solely on combining the predictions.

Hyperparameters for meta-model (in this case Random Forest) are shown below:

Hyperparameter	Value
n_estimators	100
max_depth	3

Table 4.13: Random Forest metamodel hyperparameters

Where *n_estimators* – number of trees in the forest and *max_depth* – maximum depth of each tree.

It introduced the following performance:

Metric	Value
Accuracy	65.8%

Table 4.14: Stacked model performance. Source: Author

Based on these results, it can be concluded that the stacked model with a Random Forest as the meta-learner achieved the best overall performance, showing a slight improvement over the best single model — the pruned and tuned Logistic Regression (65.7 %).

Regarding the importance of base learners used in the stacked model, the following table reflects their respective contributions:

Base Model	Contribution
Logistic Regression	36.5%
MLP	33.3%
XGBoost	30.2%

Table 4.15: Base learners' contribution. Source: Author

The pruned and tuned Logistic Regression, which also achieved the best performance among all single models, received the highest contribution.

The MLP and XGBoost models contributed slightly less, though all three base learners provided meaningful input to the final prediction.

5 Discussion

The results of this thesis demonstrate the feasibility of predicting professional Dota 2 match outcomes using only pre-game and early-game data. The best-performing model, a stacked ensemble combining Logistic Regression, Multilayer Perceptron (MLP), and XGBoost, achieved an accuracy of 65.8%. While this accuracy is well below the 93.73% reported by Yang, Qin, and Lei (2016), who used full-match data up to the 40-minute mark, and the 88–93% range achieved by Akhmedov and Phan (2021) using deep neural networks on real-time in-game features, it still captures meaningful early-game signals. It suggests that it's no need to underestimate the early game phase and teams that are not having advantage in an early game - can have negative consequences later. This makes such models potentially useful in contexts of coaching or early-game betting scenarios.

The contribution of each base model in the ensemble was as follows: Logistic Regression (36.5%), MLP (33.3%), and XGBoost (30.2%). These models were selected based on their individual performance: the optimized Logistic Regression and XGBoost models were both pruned and hyperparameter-tuned, while the MLP was used in its baseline configuration due to its performance stability. Notably, Logistic Regression showed no difference between the baseline and optimized versions (both at 65.8%), indicating limited gains from additional complexity. XGBoost improved incrementally with each tuning step, while MLP slightly declined after optimization, dropping to 64.4%. Overall, the improvements and drops were marginal, suggesting that modelling pipeline can be simplified, and new models can be introduced to have a wider comparison.

In parallel to model development, this thesis introduced a structured and scalable data processing pipeline inspired by the "Model Lake" framework proposed by Garouani, Ravat, and Valles-Parlangeau (2024). The pipeline governs data flow from raw ingestion to model deployment and tracking, supporting reproducibility and modular experimentation. Although currently limited to pre- and early-game features, the framework is designed for extensibility: additional data sources—such as real-time gameplay logs, detailed draft phase information and extended team-level historical metrics—can be incorporated without major redesign. Furthermore, the entire process can be scaled through automation, enabling continuous integration of new match data, retraining of models, and performance monitoring in production environments.

Taken together, the findings suggest that while early-game data alone does not allow for high-precision outcome prediction, it does offer consistent signals that machine learning models can detect. The results reinforce the importance of early decision-making in Dota 2 and open the door for hybrid approaches that combine interpretable early-game models with real-time updates. Future research could build on this foundation by incorporating richer temporal data, generalizing across game patches, and exploring prediction interpretability at both player and team levels.

Conclusion

The primary goal of this thesis was to explore and identify the most influential features for predicting the outcomes of professional Dota 2 matches. A secondary goal was to design and demonstrate a data governance framework capable of handling data collection from various endpoints, managing data transfers through structured layers, and applying appropriate transformations to raw data before feeding it into machine learning models.

To achieve the secondary goal, a dedicated data pipeline was developed. Raw data was first requested and transferred to a staging layer using extraction scripts. It was then transformed into a tabular format and moved to a target table, where further transformations could be applied. This final version of the dataset was used as input for machine learning models.

The final dataset was balanced in terms of the target variable and underwent exploratory analysis to identify meaningful patterns.

To fulfil the primary goal, three machine learning algorithms were selected: Multi-Layer Perceptron (MLP), XGBoost, and Logistic Regression. Two versions of the dataset were created for model training: one for MLP and XGBoost, and a second version—enhanced with additional transformations tailored for linear models—for Logistic Regression.

Initial model performance on unmodified datasets showed the following accuracy scores: 65.7% for Logistic Regression, 65.0% for MLP, and 65.3% for XGBoost.

To improve performance, feature pruning was applied. The ten least important features for Logistic Regression were dropped based on coefficient magnitude. For MLP, five features with the lowest permutation importance were removed.

For XGBoost, the ten least impactful features were identified and dropped using Shapley values. After pruning, model performance was: Logistic Regression – 65.6%, MLP – 64.9%, and XGBoost – 65.4%.

The next step was hyperparameter optimization using Randomized Search. After tuning, model performances were: Logistic Regression – 65.7%, MLP – 64.4%, and XGBoost – 65.6%.

Finally, a stacked model was implemented, combining the best-performing versions of the individual models as base learners, with a Random Forest serving as the meta-learner. The stacked model achieved an accuracy of 65.8%, outperforming all individual models introduced in the experiment.

Regarding individual feature influence, the most impactful variables were related to gold, experience, and combat—particularly net worth difference, kill difference, and experience difference. Another important category that contributed significantly was the presence of a team in the ELO rating leaderboard, which served as a proxy for overall team skill.

List of references

- Akhmedov, K., & Phan, A. H. (2021). Machine learning models for DOTA 2 outcomes prediction. arXiv preprint arXiv:2106.01782. <https://arxiv.org/abs/2106.01782>
- Altmann, A., Tološi, L., Sander, O., & Lengauer, T. (2010). Permutation importance: A corrected feature importance measure. *Bioinformatics*, 26(10), 1340–1347. <https://doi.org/10.1093/bioinformatics/btq134>
- Delello, J. A., McWhorter, R. R., Roberts, P., Dockery, H. S., De Giuseppe, T., & Corona, F. (2021). The rise of eSports. *International Journal of eSports Research*, 1(1), 1–19. <https://doi.org/10.4018/ijer.20210101.oa5>
- DreamLeague Season 24 - Winner - ESL Dota2 DreamLeague. (2025). <https://pro.eslgaming.com/dreamleague/season24/>
- Guyon, I., & Elisseeff, A. (2003). An introduction to variable and feature selection. *Journal of Machine Learning Research*, 3(Mar), 1157–1182. <https://www.jmlr.org/papers/volume3/guyon03a/guyon03a.pdf>
- Ke, C. H., Deng, H., Xu, C., Li, J., Gu, X., Yadamsuren, B., Klabjan, D., Sifa, R., Drachen, A., & Demediuk, S. (2022). DOTA 2 match prediction through deep learning team fight models. 2022 IEEE Conference on Games (CoG). https://ieeegames.org/2022/assets/papers/paper_44.pdf
- Kim, T. K. (2015). T test as a parametric statistic. *Korean Journal of Anesthesiology*, 68(6), 540. <https://doi.org/10.4097/kjae.2015.68.6.540>
- Largest overall prize pools in Esports - Esports tournament rankings:: Esports earnings. (2025). <https://www.esportsearnings.com/tournaments>
- Lundberg, S. M., & Lee, S.-I. (2017). A unified approach to interpreting model predictions. In *Advances in Neural Information Processing Systems* (Vol. 30). <https://proceedings.neurips.cc/paper/2017/hash/8a20a8621978632d76c43dfd28b67767-Abstract.html>
- Model Lake: a New Alternative for Machine Learning Models Management and Governance. (2025). https://arxiv.org/html/2503.22754v1?utm_source
- Nahhas, R. W. (2025). 9.2 Missing data mechanism | Introduction to Regression Methods for Public Health Using R. <https://www.bookdown.org/rwnahhas/RMPH/mi-mechanisms.html>
- Nalawade, B. (2024). The Essential Guide to K-Fold Cross-Validation in Machine Learning. Medium. <https://medium.com/@bididudy/the-essential-guide-to-k-fold-cross-validation-in-machine-learning-2bcb58c50578>
- Perez, S. (2024). The role of e-sports in modern recreation. *International Journal of Arts Recreation and Sports*, 3(4), 27–40. <https://doi.org/10.47941/ijars.2069>
- Perry, N., & Conley, R. (2013). How does he saw me? A recommendation engine for picking heroes in Dota 2 [CS229 course project]. Stanford University.

<https://cs229.stanford.edu/proj2013/PerryConley-HowDoesHeSawMeARecommendationEngineForPickingHeroesInDota2.pdf>

Rainio, O., Teuho, J., & Klén, R. (2024). Evaluation metrics and statistical tests for machine learning. *Scientific Reports*, 14(1). <https://doi.org/10.1038/s41598-024-56706-x>

Rasamoelina, A. D., Adjailia, F., & Sincak, P. (2020). A review of activation function for artificial neural network. *Proceedings of the 2020 International Conference on Artificial Intelligence and Computer Engineering (ICAICE)*, 1–6. https://www.researchgate.net/publication/341953192_A_Review_of_Activation_Function_for_Artificial_Neural_Network

Samuels, J. A. (2024). One-hot encoding and two-hot encoding: An introduction. *ResearchGate*. https://www.researchgate.net/publication/377159812_One-Hot_Encoding_and_Two-Hot_Encoding_An_Introduction

Soper, D. S. (2021). Greed is good: rapid hyperparameter optimization and model selection using Greedy K-Fold cross validation. *Electronics*, 10(16), 1973. <https://doi.org/10.3390/electronics10161973>

SRO, P. (2022). Esport v číslech. *esport.cz*. <https://www.esport.cz/esport-v-cislech>

The surprising history of video games and esports. (2024). <https://www.usacademicsports.com/post/history-of-esports#:~:text=While%20competitive%20gaming%20had%20been,small%20tournaments%20to%20international%20events>

Twitch Stats and Analytics - Games & Channels - SullyGnome. (2025). <https://sullygnome.com/>

Using KBinsDiscretizer to discretize continuous features. (2025). *Scikit-learn*. https://scikit-learn.org/stable/auto_examples/preprocessing/plot_discretization.html

Valve. (2024). Dota 2 Patch Notes – 7.36. <https://www.dota2.com/patches/7.36>

Valve. (2025). Dota 2 Patch Notes – 7.38. <https://www.dota2.com/patches/7.38>

Wolpert, D. H. (1992). Stacked generalization. *Neural Networks*, 5(2), 241–259. [https://doi.org/10.1016/S0893-6080\(05\)80023-1](https://doi.org/10.1016/S0893-6080(05)80023-1)

Yang, Y., Qin, T., & Lei, Y.-H. (2016). Real-time eSports match result prediction. *arXiv*. <https://doi.org/10.48550/arXiv.1701.03162>

Yao, S. (2019). For Love of the Game: A Study of Tournament Theory and Intrinsic Motivation in Dota 2.

Yuen, S., Thomson, J. D., & Don, O. (2020). Automatic player identification in Dota 2. *arXiv*. <https://doi.org/10.48550/arXiv.2008.12401>

Appendices

A. SQL Scripts for tables creation

```
--CREATE SCHEMA metadata
DROP TABLE IF EXISTS metadata.Source_Metadata;
CREATE TABLE metadata.Source_Metadata (
    source_metadata_id INT IDENTITY(1,1) NOT NULL,
    source_id INT NOT NULL,
    source_name VARCHAR(200) NOT NULL,
    endpoint VARCHAR(MAX) NOT NULL,
    endpoint_name VARCHAR(200) NOT NULL,
    parameters VARCHAR(MAX) NULL,
    created_at DATETIME DEFAULT GETDATE(),
    CONSTRAINT PK_Source_Metadata PRIMARY KEY (source_id, source_metadata_id)
);
DROP TABLE IF EXISTS metadata.ETL_Run_Metadata;
CREATE TABLE metadata.ETL_Run_Metadata (
    run_id INT IDENTITY(1,1) PRIMARY KEY,
    source_id INT NOT NULL,
    source_metadata_id INT NOT NULL,
    run_timestamp DATETIME NOT NULL DEFAULT GETDATE(),
    records_extracted INT NULL,
    records_loaded INT NULL,
    run_status VARCHAR(20) NOT NULL CHECK (run_status IN ('SUCCESS', 'FAILURE')),
    [error_message] VARCHAR(MAX) NULL,
    etl_part VARCHAR(20) NOT NULL CHECK (etl_part IN ('EXTRACT TO STAGE', 'LOAD TO TARGET',
'FEATURE ENGINEERING')),
    duration INT,
    CONSTRAINT FK_ETL_Source_Metadata FOREIGN KEY (source_id, source_metadata_id)
    REFERENCES metadata.Source_Metadata (source_id, source_metadata_id)
);
DROP TABLE IF EXISTS metadata.Data_Lineage_Metadata;
CREATE TABLE metadata.Data_Lineage_Metadata (
    lineage_id INT IDENTITY(1,1) PRIMARY KEY,
    run_id INT NOT NULL FOREIGN KEY REFERENCES metadata.ETL_Run_Metadata(run_id),
    source_table VARCHAR(100) NOT NULL,
    target_table VARCHAR(100) NOT NULL,
    lineage_timestamp DATETIME NOT NULL DEFAULT GETDATE()
);
DROP TABLE IF EXISTS metadata.Model_Training_Metadata;
CREATE TABLE metadata.Model_Training_Metadata (
    model_run_id INT IDENTITY(1,1) PRIMARY KEY,
    run_timestamp DATETIME NOT NULL DEFAULT GETDATE(),
```

```

model_name VARCHAR(100) NOT NULL,          -- e.g. dota_earlygame_v1
ml_algorithm VARCHAR(50) NOT NULL,         -- e.g. XGBoost, RandomForest, HGB
data_category VARCHAR(100) NOT NULL,       -- e.g. "in-game", "draft"

training_accuracy FLOAT,
training_roc_auc FLOAT,
training_log_loss FLOAT,

test_accuracy FLOAT,
test_roc_auc FLOAT,
test_log_loss FLOAT,

hyperparameters NVARCHAR(MAX),
steps_implemented NVARCHAR(MAX),          -- e.g. "encoding, gridsearch, boosting"

execution_seconds FLOAT,
error_flag BIT DEFAULT 0,                 -- 0 = success, 1 = failed
error_message NVARCHAR(MAX) NULL
);

```

```

DROP TABLE IF EXISTS metadata.Import_Schedule
CREATE TABLE metadata.Import_Schedule (
    import_id INT IDENTITY(1,1) PRIMARY KEY,
    endpoint_name VARCHAR(200) NOT NULL, -- like 'leagues_matches'
    stage_table VARCHAR(200) NOT NULL, -- like 'stg_leagues_matches'
    target_table VARCHAR(200) NOT NULL, -- like 'tgt_leagues_matches'
    load_type VARCHAR(10) NOT NULL CHECK (load_type IN ('FULL', 'BATCH')),
    schedule_description VARCHAR(255) NULL, -- optional: to describe when/why
    created_at DATETIME DEFAULT GETDATE(),
    updated_at DATETIME NULL
);

```

Listing A.1: Scripts used for Metadata tables creation. Source: Author

```

DROP TABLE IF EXISTS dbo.stg_heroes;
CREATE TABLE dbo.stg_heroes (
    transfer_id INT IDENTITY(1,1) PRIMARY KEY,
    response NVARCHAR(MAX) NOT NULL,
    source_id INT NOT NULL,
    source_metadata_id INT NOT NULL, -- Added this column
    created_at DATETIME NOT NULL DEFAULT GETDATE(),
    response_status VARCHAR(50),
    CONSTRAINT FK_StgHeroes_SourceMetadata FOREIGN KEY (source_id, source_metadata_id)
        REFERENCES metadata.Source_Metadata (source_id, source_metadata_id)
);

```

```

DROP TABLE IF EXISTS dbo.stg_matches;
CREATE TABLE dbo.stg_matches (
    transfer_id INT IDENTITY(1,1) PRIMARY KEY,
    response NVARCHAR(MAX) NOT NULL,
    source_id INT NOT NULL,
    source_metadata_id INT NOT NULL, -- Added this column
    created_at DATETIME NOT NULL DEFAULT GETDATE(),
    response_status VARCHAR(50),
    CONSTRAINT FK_StgMatches_SourceMetadata FOREIGN KEY (source_id, source_metadata_id)
        REFERENCES metadata.Source_Metadata (source_id, source_metadata_id)
);

DROP TABLE IF EXISTS dbo.stg_leagues_matches;
CREATE TABLE dbo.stg_leagues_matches (
    transfer_id INT IDENTITY(1,1) PRIMARY KEY,
    response NVARCHAR(MAX) NOT NULL,
    source_id INT NOT NULL,
    source_metadata_id INT NOT NULL, -- Added this column
    created_at DATETIME NOT NULL DEFAULT GETDATE(),
    response_status VARCHAR(50),
    CONSTRAINT FK_StgLeaguesMatches_SourceMetadata FOREIGN KEY (source_id, source_metadata_id)
        REFERENCES metadata.Source_Metadata (source_id, source_metadata_id)
);

```

Listing A.2: Scripts used for Stage tables creation. Source: Author

```

DROP TABLE IF EXISTS tgt_heroes;

CREATE TABLE tgt_heroes (
    hero_id INT PRIMARY KEY,
    hero_name VARCHAR(100) NOT NULL,
    localized_name VARCHAR(100) NOT NULL,
    primary_attr VARCHAR(10),
    attack_type VARCHAR(20),
    roles VARCHAR(255),
    updated_at DATETIME DEFAULT GETDATE(),
    loaded_at DATETIME DEFAULT GETDATE()
);

DROP TABLE IF EXISTS dbo.tgt_leagues_matches

CREATE TABLE dbo.tgt_leagues_matches (
    match_id BIGINT PRIMARY KEY,
    league_id INT,
    start_time DATETIME,
    duration INT,
    radiant_team_id INT NULL,
    radiant_team_name VARCHAR(255) NULL,
    dire_team_id INT NULL,

```

```

dire_team_name VARCHAR(255) NULL,
radiant_win BIT,
radiant_score INT,
dire_score INT,
series_id INT NULL,
series_type INT NULL,
professionality VARCHAR(10), -- "PRO" or "PUBLIC"
tier VARCHAR(50) NULL,
updated_at DATETIME DEFAULT GETDATE(),
        loaded_at DATETIME DEFAULT GETDATE()
);
DROP TABLE IF EXISTS dbo.tgt_matches
CREATE TABLE dbo.tgt_matches (
    version VARCHAR(50),
    match_id BIGINT PRIMARY KEY,
    draft_timings NVARCHAR(MAX),
    teamfights NVARCHAR(MAX),
    objectives NVARCHAR(MAX),
    radiant_gold_adv NVARCHAR(MAX),
    radiant_xp_adv NVARCHAR(MAX),
    players NVARCHAR(MAX),
    leagueid INT,
    start_time DATETIME,
    duration INT,
    series_id INT,
    series_type INT,
    cluster INT,
    radiant_win BIT,
    pre_game_duration INT,
    match_seq_num BIGINT,
    tower_status_radiant INT,
    tower_status_dire INT,
    barracks_status_radiant INT,
    barracks_status_dire INT,
    first_blood_time INT,
    lobby_type INT,
    human_players INT,
    game_mode INT,
    flags INT,
    engine INT,
    radiant_score INT,
    dire_score INT,
    radiant_team_id INT,
    radiant_name VARCHAR(255),
    radiant_team_complete INT,
    dire_team_id INT,

```

```

dire_name VARCHAR(255),
dire_team_complete INT,
radiant_captain BIGINT,
dire_captain BIGINT,
picks_bans NVARCHAR(MAX),
patch INT,
region INT,
comeback BIT,
stomp BIT,
league_leagueid INT,
league_tier VARCHAR(50),
league_name VARCHAR(255),
radiant_team_tag VARCHAR(50),
dire_team_tag VARCHAR(50),
loaded_at DATETIME DEFAULT GETDATE(),
updated_at DATETIME NULL
);

```

A.3: Scripts used for Target tables creation. Source: Author

```

-- matches that having non-empty 'draft_timings' data
SELECT match_id, draft_timings, radiant_win
INTO dbo.tgt_draft_data
FROM dbo.tgt_matches
WHERE draft_timings <> '';

```

A.4: Script used to filter out invalid draft stage data. Source: Author

B. Python script for data processing example

```
import pyodbc
import requests
import json
import time
from datetime import datetime

API_DELAY = 0.07 # ~850 calls/min
ENDPOINT_NAME = 'matches_details'

with open('secrets/datasources_keys.json') as f:
    keys = json.load(f)
with open('secrets/db_creds.json') as f:
    db_creds = json.load(f)

conn = pyodbc.connect(
    f'DRIVER={{db_creds["DRIVER"]}};'
    f'SERVER={db_creds["SERVER"]};'
    f'DATABASE={db_creds["DATABASE"]};'
    f'UID={db_creds["UID"]};'
    f'PWD={db_creds["PASSWORD"]}'
)
cursor = conn.cursor()

print('✅ Starting extraction script for matches_details...')

# 1) Ensure Source Metadata
create_source_metadata_sql = """
IF NOT EXISTS (
    SELECT 1 FROM metadata.Source_Metadata
    WHERE source_id = 1 AND endpoint_name = 'matches_details'
)
BEGIN
    INSERT INTO metadata.Source_Metadata (
        source_id, source_name, endpoint, endpoint_name, parameters, created_at
    )
    VALUES (
        1,
        'OpenDota',
        'https://api.opendota.com/api/matches/{match_id}',
        'matches_details',
```

```

        'match_id',
        GETDATE()
    );
END
'''

cursor.execute(create_source_metadata_sql)
conn.commit()

# 2) Ensure Import Schedule for matches_details
create_import_schedule_sql = '''
IF NOT EXISTS (
    SELECT 1 FROM metadata.Import_Schedule
    WHERE endpoint_name = 'matches_details'
)
BEGIN
    INSERT INTO metadata.Import_Schedule (
        source_metadata_id,
        endpoint_name,
        stage_table,
        target_table,
        load_type,
        schedule_description,
        created_at
    )
    SELECT
        source_metadata_id,
        'matches_details',
        'stg_matches',
        'tgt_matches',
        'BATCH',
        'Auto-created schedule for matches details',
        GETDATE()
    FROM metadata.Source_Metadata
    WHERE endpoint_name = 'matches_details';
END
'''

cursor.execute(create_import_schedule_sql)
conn.commit()

# 3) Load schedule info to find the staging table
get_schedule_sql = '''
SELECT load_type, stage_table, target_table
FROM metadata.Import_Schedule
WHERE endpoint_name = ?
'''

cursor.execute(get_schedule_sql, (ENDPOINT_NAME,))

```

```

row_schedule = cursor.fetchone()
if not row_schedule:
    raise Exception(f❌ No Import_Schedule entry for endpoint="{ENDPOINT_NAME}")

planned_load_type, staging_table, target_table = row_schedule

# 4) Load Source Metadata IDs
get_source_metadata_sql = """
SELECT source_id, source_metadata_id
FROM metadata.Source_Metadata
WHERE endpoint_name = ?
"""

cursor.execute(get_source_metadata_sql, (ENDPOINT_NAME,))
row_metadata = cursor.fetchone()
if not row_metadata:
    raise Exception(f❌ No Source_Metadata entry found for endpoint="{ENDPOINT_NAME}")

source_id, source_metadata_id = row_metadata

print(f✅ Staging table is "{staging_table}" (target is "{target_table}" but we do not load here).')

# 5) For extraction, we do NOT care about target load type logic
# because we only populate staging.

# === Fetch match_ids from your leagues table
fetch_leagues_sql = 'SELECT match_id, tier FROM dbo.tgt_leagues_matches'
cursor.execute(fetch_leagues_sql)
matches_list = cursor.fetchall()

api_key = keys['OpenDota']
endpoint_template = 'https://api.opendota.com/api/matches/{match_id}'
EXTRACT_ETL_PART = 'EXTRACT TO STAGE'
records_extracted = 0

def clean_match_data(data):
    keys_to_remove = [
        'chat', 'replay_salt', 'radiant_logo', 'dire_logo',
        'metadata', 'replay_url', 'league_ticket', 'league_banner'
    ]
    patterns_to_remove = ['cosmetics', 'od_data', 'all_word_counts']

    for key in keys_to_remove:
        data.pop(key, None)
    for key in list(data.keys()):
        if any(pattern in key for pattern in patterns_to_remove):
            data.pop(key, None)

```



```

if 'radiant_team' in data:
    data['radiant_team'].pop('logo_url', None)
    data['radiant_team'].pop('name', None)
    data['radiant_team'].pop('team_id', None)
if 'dire_team' in data:
    data['dire_team'].pop('logo_url', None)
    data['dire_team'].pop('name', None)
    data['dire_team'].pop('team_id', None)
return data

for idx, (match_id_val, tier_val) in enumerate(matches_list, start=1):
    try:
        endpoint = endpoint_template.replace('{match_id}', str(match_id_val))
        response = requests.get(endpoint, params={'api_key': api_key})
        response.raise_for_status()

        match_data = response.json()
        match_data = clean_match_data(match_data)

        # Insert JSON into staging
        insert_staging_sql = f"""
INSERT INTO dbo.{staging_table} (
    response, source_id, source_metadata_id, created_at, response_status
)
VALUES (?, ?, ?, GETDATE(), ?)
"""

        cursor.execute(insert_staging_sql, (
            json.dumps(match_data),
            source_id,
            source_metadata_id,
            response.status_code
        ))
        conn.commit()

        # Log extraction success
        etl_log_extract_sql = """
INSERT INTO metadata.ETL_Run_Metadata (
    source_id, source_metadata_id, run_timestamp, records_extracted,
    run_status, etl_part, error_message
)
VALUES (?, ?, GETDATE(), 1, 'SUCCESS', ?, NULL)
"""

        cursor.execute(etl_log_extract_sql, (source_id, source_metadata_id, EXTRACT_ETL_PART))
        conn.commit()

```

```

# Data lineage: from source => staging (some folks do target_table here too,
# but it's referencing the final target. We'll keep same approach as you had.)
data_lineage_sql = ""
INSERT INTO metadata.Data_Lineage_Metadata (
    run_id, source_table, target_table, lineage_timestamp
)
VALUES (
    (SELECT MAX(run_id) FROM metadata.ETL_Run_Metadata WHERE source_metadata_id = ?),
    ?,
    ?,
    GETDATE()
)
"""

    cursor.execute(data_lineage_sql, (source_metadata_id, staging_table, target_table))
    conn.commit()

    records_extracted += 1
    print(f'✅ Extracted match {match_id_val} [{idx}/{len(matches_list)}]')
    time.sleep(API_DELAY)

except Exception as e:
    err_msg = str(e)[:2000]
    print(f'❌ Extraction error for match {match_id_val}: {err_msg}')

    etl_extract_fail_sql = ""
INSERT INTO metadata.ETL_Run_Metadata (
    source_id, source_metadata_id, run_timestamp, records_extracted,
    run_status, etl_part, error_message
)
VALUES (?, ?, GETDATE(), 0, 'FAILURE', ?, ?)
"""

    cursor.execute(etl_extract_fail_sql, (source_id, source_metadata_id, EXTRACT_ETL_PART, err_msg))
    conn.commit()
    continue

print(f'✅ Extraction completed. Inserted {records_extracted} records into staging = {staging_table}')

cursor.close()
conn.close()
print('✅ Script finished its run!')

```

B.1: Script used for extracting raw data from “matches” endpoint (aka. “matches_details”) to “stg_matches_details”

C. Endpoints list

Endpoint name	Purpose:	Endpoint
Matches/Matches details	Matches detailed information (main source of data)	https://api.opendota.com/api/matches/{match_id}
Heroes	Heroes basic characteristics	https://api.opendota.com/api/heroes
Leagues matches	High-level matches information for each league	https://api.opendota.com/api/leagues/{league_id}/matches
Leagues	Gives list of all leagues within OpenDota database	https://api.opendota.com/api/leagues

C.1: Table of endpoints. Source: Author

D. Dataset preview

match_id	77534204	77535450	77536514	77540907	77543421	77545695	77547190	77558327	77559976	77561279
radiant_win	1	1	1	1	1	0	1	0	0	1
first_blood_time	83	556	43	14	0	130	2	210	0	222
first_bottle_team	Radiant	Dire	Radiant	Radiant	Dire	Dire	Dire	Radiant	Radiant	Radiant
first_boots_team	Radiant	Radiant	Radiant	Radiant	Radiant	Dire	Radiant	Dire	Dire	Radiant
first_safelane_tower_destroyed_by	Dire	Dire	Radiant	Radiant	Radiant	Radiant	Dire	Dire	Radiant	Dire
first_mid_tower_destroyed_by	Dire	Dire	Dire	Dire	Radiant	Dire	Dire	Dire	Dire	Dire
first_offlane_tower_destroyed_by	Radiant	Radiant	Radiant	Radiant	Radiant	Dire	Dire	Dire	Dire	Dire
first_safelane_tower_time	2194.0	1273.0	1615.0	1940.0	1400.0	992.0	1509.0	1239.0	794.0	1338.0
first_mid_tower_time	2248.0	1213.0	1294.0	2524.0	1622.0	780.0	1379.0	1672.0	1268.0	1266.0
first_offlane_tower_time	1000.0	702.0	1835.0	1553.0	1567.0	1518.0	1141.0	1330.0	1783.0	1257.0
courier_kill_diff	2	0	0	0	-2	1	-1	1	1	0
courier_gold_diff	60	0	0	0	-70	25	-30	-40	70	0
frag_diff_10_min	7	-1	-1	1	-5	-2	-1	2	-3	5
total_frag	44	19	46	9	69	37	54	62	45	43
networth_diff_10min	2013	1485	-2134	-752	-1473	-888	-163	161	762	2835
consumable_gold_diff	-70	-35	485	275	-920	705	-470	-405	530	245
lotus_picks_diff	0	0	0	0	0	0	0	0	0	0
total_gold_spent_diff	2013	1485	-2134	-752	-1473	-888	-163	161	762	2835
obs_placed_diff	2	0	-1	-1	-1	0	0	1	0	2
sen_placed_diff	0	1	-1	-1	1	-1	0	1	0	-1
obs_destroyed_diff	1	0	0	2	3	0	1	1	0	1
wisdom_runes_diff	0	0	0	0	0	0	2	0	0	0
sum_denies_diff	26	19	-19	-7	9	20	25	17	-3	24
sum_lasthits_diff	21	38	-39	-8	-30	11	30	16	-8	28
str_heroes_diff	-2	1	-1	-2	2	-1	0	1	0	0
agi_heroes_diff	1	0	1	-1	-2	0	-1	-1	2	2
int_heroes_diff	2	-1	-1	1	-1	0	1	1	-2	-2
all_heroes_diff	-1	0	1	2	1	1	0	-1	0	0
drums_presence	None	None	None	None	None	None	None	None	None	None
vladmir_presence	None	None	None	None	None	None	None	None	None	None
roamer_diff	0	1	0	0	-1	0	1	0	1	0
first_pick	Radiant	Radiant	Dire	Dire	Radiant	Dire	Dire	Dire	Dire	Dire
radiant_elo	1547.38	1455.86	1455.86	1333.88	1333.88	1545.06	1547.38	1381.78	1333.88	1381.78
dire_elo	1455.86	1547.38	1547.38	1237.96	1237.96	1547.38	1545.06	1333.88	1381.78	1333.88
xp_diff	2373.0	1834.0	143.0	684.0	-1078.0	-102.0	1826.0	33.0	86.0	2296.0
bounty_rune_diff	-2.0	-1.0	-6.0	-2.0	2.0	-4.0	-5.0	-3.0	5.0	4.0
first_bounty_advantage	Dire	Dire	Dire	Dire	Radiant	Dire	Dire	Dire	Radiant	Radiant
water_rune_diff	0.0	0.0	0.0	0.0	0.0	1.0	0.0	0.0	0.0	0.0

D.1: Transposed raw dataset preview – first 10 rows with all features. Source: Author

E. Features description

Name	Data Type	Feature Group	Purpose
match_id	Integer	In-game	Unique identifier of the match
radiant_win	Integer	In-game	Match outcome (1 = Radiant won, 0 = Dire won)
first_blood_time	Integer	In-game	Time in seconds when the first kill happened
first_bottle_team	String	In-game	Team that bought the first bottle item
first_boots_team	String	In-game	Team that bought the first boots of speed
courier_kill_diff	Integer	In-game	Difference in courier kills between teams
courier_gold_diff	Integer	In-game	Difference in gold earned by courier kills between teams
frag_diff_10_min	Integer	In-game	Kill difference between teams at 10 minutes
networth_diff_10min	Integer	In-game	Net worth difference at 10 minutes
consumable_gold_diff	Integer	In-game	Difference in gold spent on consumable items
lotus_picks_diff	Integer	In-game	Difference in Greater Healing Lotus pickups
total_gold_spent_diff	Integer	In-game	Difference in total gold spent
obs_placed_diff	Integer	In-game	Difference in observer wards placed
sen_placed_diff	Integer	In-game	Difference in sentry wards placed
obs_destroyed_diff	Integer	In-game	Difference in observer wards destroyed
wisdom_runes_diff	Integer	In-game	Difference in wisdom runes collected
sum_denies_diff	Integer	In-game	Difference in total denies between teams
sum_lasthits_diff	Integer	In-game	Difference in total last hits between teams

str_heroes_diff	Integer	Pre-game	Difference in strength heroes picked
agi_heroes_diff	Integer	Pre-game	Difference in agility heroes picked
int_heroes_diff	Integer	Pre-game	Difference in intelligence heroes picked
all_heroes_diff	Integer	Pre-game	Difference in universal heroes picked
drums_presence	String	In-game	Which team bought Drums of Endurance first
vladmir_presence	String	In-game	Which team bought Vladimir's Offering first
roamer_diff	Integer	In-game	Difference in number of roaming heroes
first_pick	String	Pre-game	Team that had first pick in draft
radiant_elo	Float	Pre-game	ELO rating of Radiant team
dire_elo	Float	Pre-game	ELO rating of Dire team
xp_diff	Integer	In-game	In-game experience point difference between teams
bounty_rune_diff	Integer	In-game	Difference in bounty runes collected
first_bounty_advantage	String	In-game	Team that gained advantage at first bounty runes
water_rune_diff	Integer	In-game	Difference in water runes collected
first_offlane_tower_destroyed_by	String	In-game	Team that destroyed the first offlane T1 tower
first_offlane_tower_time	Float	In-game	Time when first offlane T1 tower was destroyed
first_mid_tower_destroyed_by	String	In-game	Team that destroyed the first midlane T1 tower
first_mid_tower_time	Float	In-game	Time when first midlane T1 tower was destroyed
first_safelane_tower_destroyed_by	String	In-game	Team that destroyed the first safelane T1 tower
first_safelane_tower_time	Float	In-game	Time when first safelane T1 tower was destroyed

E.1: Features detailed description table. Source: Author